

TUGAS AKHIR - EC234701

PENERAPAN MODEL MACHINE LEARNING UNTUK MENINGKATKAN EFISIENSI ALGORITMA CLOCK PADA CACHE REPLACEMENT

Muhammad Haekal Muhyidin Al-Araby

NRP 5024 22 1030

Dosen Pembimbing

Reza Fuad Rachmadi, S.T., M.T., Ph.D

NIP 19850403 201212 1 001

Dr. Arief Kurniawan, S.T., M.T

NIP 19740907 200212 1 001

Program Studi Strata 1 (S1) Teknik Komputer

Departemen Teknik Komputer

Fakultas Teknologi Elektro dan Informatika Cerdas

Institut Teknologi Sepuluh Nopember

Surabaya

2026



TUGAS AKHIR - EC234701

**PENERAPAN MODEL MACHINE LEARNING UNTUK
MENINGKATKAN EFISIENSI ALGORITMA CLOCK
PADA CACHE REPLACEMENT**

Muhammad Haekal Muhyidin Al-Araby

NRP 5024 22 1030

Dosen Pembimbing

Reza Fuad Rachmadi, S.T., M.T., Ph.D

NIP 19850403 201212 1 001

Dr. Arief Kurniawan, S.T., M.T

NIP 19740907 200212 1 001

Program Studi Strata 1 (S1) Teknik Komputer

Departemen Teknik Komputer

Fakultas Teknologi Elektro dan Informatika Cerdas

Institut Teknologi Sepuluh Nopember

Surabaya

2026

[Halaman ini sengaja dikosongkan]



FINAL PROJECT - EC234701

***APPLICATION OF MACHINE LEARNING MODELS TO
IMPROVE THE EFFICIENCY OF THE CLOCK
ALGORITHM IN CACHE REPLACEMENT***

Muhammad Haekal Muhyidin Al-Araby

NRP 5024 22 1030

Advisor

Reza Fuad Rachmadi, S.T., M.T., Ph.D

NIP 19850403 201212 1 001

Dr. Arief Kurniawan, S.T., M.T

NIP 19740907 200212 1 001

Undergraduate Study Program of Computer Engineering

Department of Computer Engineering

Faculty of Electrical Engineering and Intelligent Informatics

Sepuluh Nopember Institute of Technology

Surabaya

2026

[Halaman ini sengaja dikosongkan]

LEMBAR PENGESAHAN

PENERAPAN MODEL MACHINE LEARNING UNTUK MENINGKATKAN EFISIENSI ALGORITMA CLOCK PADA CACHE REPLACEMENT

TUGAS AKHIR

Diajukan untuk memenuhi salah satu syarat
memperoleh gelar Sarjana Teknik pada
Program Studi S-1 Teknik Komputer
Departemen Teknik Komputer
Fakultas Teknologi Elektro dan Informatika Cerdas
Institut Teknologi Sepuluh Nopember

Oleh: **Muhammad Haekal Muhyidin Al-Araby**
NRP. 5024 22 1030

Disetujui oleh Tim Penguji Tugas Akhir:

Reza Fuad Rachmadi, S.T., M.T., Ph.D
NIP: 19850403 201212 1 001

(Pembimbing I)

.....

Dr. Arief Kurniawan, S.T., M.T
NIP: 19740907 200212 1 001

(Pembimbing II)

.....

Dr. Eko Mulyanto Yuniarno, S.T., M.T.
NIP: 19680601 199512 1 009

(Penguji I)

.....

Dr. Diah Puspito Wulandari, S.T., M.Sc.
NIP: 19801219 200501 2 001

(Penguji II)

.....

Yusril Izza, S.T., M.Comp.
NIP: 1992202511059

(Penguji III)

.....

Mengetahui,
Kepala Departemen Teknik Komputer FTEIC - ITS

Dr. Arief Kurniawan, S.T., M.T.
NIP. 19740907 200212 1 001

SURABAYA
Januari, 2026

[Halaman ini sengaja dikosongkan]

APPROVAL SHEET

APPLICATION OF MACHINE LEARNING MODELS TO IMPROVE THE EFFICIENCY OF THE CLOCK ALGORITHM IN CACHE REPLACEMENT

FINAL PROJECT

Submitted to fulfill one of the requirements
for obtaining a degree Bachelor of Engineering at
Undergraduate Study Program of Computer Engineering
Department of Computer Engineering
Faculty of Electrical Engineering and Intelligent Informatics
Sepuluh Nopember Institute of Technology

By: **Muhammad Haekal Muhyidin Al-Araby**
NRP. 5024 22 1030

Approved by Final Project Examiner Team:

Reza Fuad Rachmadi, S.T., M.T., Ph.D
NIP: 19850403 201212 1 001

(Advisor I)

.....

Dr. Arief Kurniawan, S.T., M.T
NIP: 19740907 200212 1 001

(Co-Advisor II)

.....

Dr. Eko Mulyanto Yuniarno, S.T., M.T.
NIP: 19680601 199512 1 009

(Examiner I)

.....

Dr. Diah Puspito Wulandari, S.T., M.Sc.
NIP: 19801219 200501 2 001

(Examiner II)

.....

Yusril Izza, S.T., M.Comp.
NIP: 1992202511059

(Examiner III)

.....

Acknowledged,
Head of Computer Engineering Department ELECTICS - ITS

Dr. Arief Kurniawan, S.T., M.T.
NIP. 19740907 200212 1 001

SURABAYA
January, 2026

[Halaman ini sengaja dikosongkan]

PERNYATAAN ORISINALITAS

Yang bertanda tangan dibawah ini:

Nama Mahasiswa / NRP : Muhammad Haekal Muhyidin Al-Araby / 5024 22 1030
Departemen : Teknik Komputer
Dosen Pembimbing / NIP : Reza Fuad Rachmadi, S.T., M.T., Ph.D / 19850403 201212 1 001

Dengan ini menyatakan bahwa Tugas Akhir dengan judul "PENERAPAN MODEL MACHINE LEARNING UNTUK MENINGKATKAN EFISIENSI ALGORITMA CLOCK PADA CACHE REPLACEMENT" adalah hasil karya sendiri, bersifat orisinal, dan ditulis dengan mengikuti kaidah penulisan ilmiah.

Bilamana di kemudian hari ditemukan ketidaksesuaian dengan pernyataan ini, maka saya bersedia menerima sanksi sesuai dengan ketentuan yang berlaku di Institut Teknologi Sepuluh Nopember.

Surabaya, January 2026

Mengetahui
Dosen Pembimbing

Mahasiswa

Reza Fuad Rachmadi, S.T., M.T., Ph.D
NIP. 19850403 201212 1 001

Muhammad Haekal Muhyidin Al-Araby
NRP. 5024 22 1030

[Halaman ini sengaja dikosongkan]

STATEMENT OF ORIGINALITY

The undersigned below:

Name of student / NRP : Muhammad Haekal Muhyidin Al-Araby / 5024 22 1030
Department : Computer Engineering
Advisor / NIP : Reza Fuad Rachmadi, S.T., M.T., Ph.D / 19850403 201212 1
001

Hereby declared that the Final Project with the title of "*APPLICATION OF MACHINE LEARNING MODELS TO IMPROVE THE EFFICIENCY OF THE CLOCK ALGORITHM IN CACHE REPLACEMENT*" is the result of my own work, is original, and is written by following the rules of scientific writing.

If in future there is a discrepancy with this statement, then I am willing to accept sanctions in accordance with provisions that apply at Sepuluh Nopember Institute of Technology.

Surabaya, January 2026

Acknowledged
Advisor

Student

Reza Fuad Rachmadi, S.T., M.T., Ph.D
NIP. 19850403 201212 1 001

Muhammad Haekal Muhyidin Al-Araby
NRP. 5024 22 1030

[Halaman ini sengaja dikosongkan]

ABSTRAK

Nama Mahasiswa : Muhammad Haekal Muhyidin Al-Araby
Judul Tugas Akhir : PENERAPAN MODEL MACHINE LEARNING UNTUK
MENINGKATKAN EFISIENSI ALGORITMA CLOCK PADA
CACHE REPLACEMENT
Pembimbing : 1. Reza Fuad Rachmadi, S.T., M.T., Ph.D
2. Dr. Arief Kurniawan, S.T., M.T

Penelitian ini berfokus pada pengembangan dan analisis algoritma *cache eviction* Clock berbasis *machine learning*. Studi ini bertujuan untuk meningkatkan kinerja algoritma Clock dengan memanfaatkan model *supervised learning* dalam proses pengambilan keputusan untuk *cache eviction*. Simulasi dilakukan menggunakan perangkat lunak kustom berbasis Lib-CacheSim dengan skenario akses data *trace replay*. Kinerja algoritma yang dikembangkan dianalisis menggunakan metrik seperti *hit ratio*, *miss ratio*, dan jumlah promosi. Hasil penelitian menunjukkan bahwa mengintegrasikan *machine learning* ke dalam algoritma Clock dapat meningkatkan kinerja dengan mengurangi jumlah promosi *cache*, menjadikannya lebih optimal dibandingkan algoritma Clock konvensional.

Kata Kunci: *Cache, Clock, LRU, FIFO, Sistem, Machine Learning, Supervised Learning*

[Halaman ini sengaja dikosongkan]

ABSTRACT

Name : Muhammad Haekal Muhyidin Al-Araby
Title : *APPLICATION OF MACHINE LEARNING MODELS TO IMPROVE THE EFFICIENCY OF THE CLOCK ALGORITHM IN CACHE REPLACEMENT*
Advisors : 1. Reza Fuad Rachmadi, S.T., M.T., Ph.D
2. Dr. Arief Kurniawan, S.T., M.T

This research focuses on the development and analysis of a machine learning-based Clock cache eviction algorithm. The study aims to improve the performance of the Clock algorithm by utilizing a supervised learning model in the decision-making process for cache eviction. Simulations were conducted using custom software based on LibCacheSim with a trace replay data access scenario. The performance of the developed algorithm was analyzed using metrics such as hit ratio, miss ratio, and the number of promotions. The results indicate that integrating machine learning into the Clock algorithm can enhance performance by reducing the number of cache promotions, making it more optimal than the conventional Clock algorithm.

Keywords: *Cache, Clock, LRU, FIFO, System, Machine Learning, Supervised Learning*

[Halaman ini sengaja dikosongkan]

KATA PENGANTAR

Puji syukur penulis panjatkan kehadiran Tuhan Yang Maha Esa, yang atas rahmat dan karunia-Nya telah memberikan kekuatan dan kelancaran sehingga penulis dapat menyelesaikan Laporan Tugas Akhir yang berjudul **"PENERAPAN MODEL MACHINE LEARNING UNTUK MENINGKATKAN EFISIENSI ALGORITMA CLOCK PADA CACHE REPLACEMENT"** dengan baik dan tepat waktu.

Penelitian dan penyusunan Laporan Tugas Akhir ini merupakan salah satu syarat untuk memperoleh gelar Sarjana Teknik pada Program Studi Teknik Komputer, Departemen Teknik Komputer, Fakultas Teknologi Elektro dan Informatika Cerdas, Institut Teknologi Sepuluh Nopember. Proses ini tidak akan berjalan lancar tanpa adanya bimbingan, dukungan, dan doa dari berbagai pihak. Oleh karena itu, pada kesempatan ini, penulis ingin mengucapkan terima kasih yang tulus kepada:

1. Keluarga tercinta, terutama Ayah, Ibu, dan seluruh saudara, yang telah memberikan dukungan moral, materiel, serta doa yang tiada henti sepanjang perjalanan studi penulis.
2. Bapak Reza Fuad Rachmadi, S.T., M.T., Ph.D, selaku Dosen Pembimbing I, dan Bapak Dr. Arief Kurniawan, S.T., M.T, selaku Dosen Pembimbing II, yang telah dengan sabar memberikan bimbingan, arahan, dan masukan yang sangat berharga sejak awal hingga akhir penyusunan tugas akhir ini.
3. Bapak Dr. Arief Kurniawan, S.T., M.T, selaku Kepala Departemen Teknik Komputer, serta seluruh jajaran dosen dan staf di lingkungan departemen yang telah memberikan bekal ilmu dan kemudahan administrasi selama masa perkuliahan.
4. Seluruh sahabat dan rekan-rekan seperjuangan di Departemen Teknik Komputer yang telah menjadi teman diskusi, memberikan semangat, dan saling membantu dalam suka maupun duka selama menempuh pendidikan.
5. Semua pihak yang tidak dapat penulis sebutkan satu per satu, yang telah memberikan kontribusi dan dukungan dalam penyelesaian laporan ini.

Penulis menyadari bahwa Laporan Tugas Akhir ini masih jauh dari kesempurnaan. Oleh karena itu, segala bentuk kritik dan saran yang membangun akan penulis terima dengan lapang dada demi perbaikan di masa mendatang. Akhir kata, semoga laporan ini dapat memberikan manfaat dan kontribusi bagi perkembangan ilmu pengetahuan, khususnya di bidang *caching system* dan *machine learning*.

Surabaya, Januari 2026

Muhammad Haekal Muhyidin Al-Araby

[Halaman ini sengaja dikosongkan]

DAFTAR ISI

[Halaman ini sengaja dikosongkan]

DAFTAR GAMBAR

[Halaman ini sengaja dikosongkan]

DAFTAR TABEL

[Halaman ini sengaja dikosongkan]

BAB I

PENDAHULUAN

Penelitian ini didorong oleh kemajuan pesat dalam teknologi komputasi modern, yang menuntut adanya teknik-teknik yang lebih canggih untuk manajemen *cache* objek. Dalam beberapa tahun terakhir, *machine learning* telah menjadi topik yang sangat populer dalam riset ilmu komputer. Berbagai studi telah dilakukan untuk memahami sejauh mana kapabilitas *machine learning* dapat diterapkan dalam bidang *caching* **Herodotou2019AutoCache**, **Sethumurugan2021Designing**, **Sun2022NCache**, **Shuja2021Applying**, **Wang2018Efficient**, **Berger2018Towards**. Penelitian ini secara spesifik menyelidiki bagaimana *machine learning* dapat diimplementasikan dalam sistem *cache* untuk meningkatkan efisiensi algoritma populer seperti CLOCK terhadap distribusi zipfian **Breslau1999Web**.

1.1 Latar Belakang

Perkembangan teknologi komputasi telah melaju dengan kecepatan yang luar biasa selama dekade terakhir. Pada masa-masa awal, komputer terbatas pada prosesor *single-core* **Liptay1968Structural**, **Turing2004Lecture**, **Shima2005The**, **Bck1997Evolutionary**. Keterbatasan ini memaksa para insinyur dan pemrogram untuk merancang perangkat lunak yang dioptimalkan untuk eksekusi *single-threaded*. Seiring waktu, ketika peningkatan kinerja mulai melambat karena batasan fisik dan termal, industri beralih ke arsitektur *multi-core* **Papadopoulos1991Multithreading**, **Herlihy2006The**, **Borkenhagen2000A**, **Abdallah2017Advanced** dan paralel **Sodan2010Parallelism**, **Kim2016Performance**, **Eickemeyer1996Evaluation**. Transisi ini memperkenalkan tantangan baru dalam desain perangkat lunak, menuntut pengembang untuk mengadopsi model pemrograman konkuren dan paralel untuk memanfaatkan sepenuhnya kapabilitas perangkat keras modern. Akibatnya, pemahaman dan optimalisasi aplikasi *multi-threaded* telah menjadi aspek penting dalam riset dan pengembangan komputasi modern.

Caching telah lama menjadi bagian integral dari sistem komputer. Mekanisme ini memainkan peran krusial dalam meningkatkan kinerja sistem dengan menyimpan data yang sering diakses di lokasi penyimpanan yang lebih cepat **Belady1966A**, **Wilkes1965Slave**, **Liptay1968Structural**. Selama bertahun-tahun, berbagai strategi *caching* telah dikembangkan untuk mengoptimalkan efisiensi pengambilan data di berbagai lingkungan, mulai dari sistem operasi dan server web hingga sistem terdistribusi dan infrastruktur *cloud* **Zhang2012A**, **Liu2022Adaptive**, **Zhang2016HDCache**.

Arsitektur *multi-core*, yang telah menjadi standar industri, juga menghadirkan tantangan signifikan bagi teknik *caching* yang ada **Eklv2011Fast**, **Calandrino2009On**, **Seo2009Design**. Dalam lingkungan seperti ini, beberapa inti prosesor sering kali mengakses dan memodifikasi data *cache* yang sama secara bersamaan, yang dapat menyebabkan kondisi balapan (*race conditions*) dan inkonsistensi data **Herlihy2006The**. Untuk menjaga kebenaran data, sebagian besar implementasi *cache* tradisional menekankan penggunaan kunci (*locks*) selama operasi penyisipan, promosi, dan modifikasi **Vera2003Data**, **Hu1998A**, **Mohan2005Efficient**. Namun, penggunaan *lock* yang berlebihan dapat menimbulkan persaingan (*contention*) dan mengurangi paralelisme, yang pada akhirnya membatasi kinerja sistem pada prosesor *multi-core* modern.

Berbagai kebijakan penggantian *cache* (*cache replacement policies*) telah diusulkan untuk mencapai tujuan desain yang berbeda. Kebijakan **Least Recently Used (LRU)** umum digunakan karena kemampuannya untuk mencapai *miss ratio* yang rendah dengan mempertahankan data yang sering diakses **Fricker2012A, ONeil1993The, ONeil1999An**. Namun, LRU memerlukan pembaruan dan sinkronisasi yang sering untuk memelihara informasi kebaruan data secara presisi, yang meningkatkan *overhead* dari *locking*. Di sisi lain, kebijakan yang lebih sederhana seperti **First-In First-Out (FIFO)** sering kali lebih disukai karena skalabilitasnya dan biaya sinkronisasi yang lebih rendah, karena hampir tidak memerlukan *locking* **Dan1990An, Moir2005Using, Molnar2002A**.

Untuk mengatasi keterbatasan LRU dan FIFO, algoritma **CLOCK** dikembangkan sebagai aproksimasi dari LRU yang tetap mempertahankan skalabilitas tinggi **Tang1976Cache**. **CLOCK** menggunakan *buffer* melingkar (*circular buffer*) dari entri *cache*, di mana setiap entri memiliki bit referensi yang menandakan akses terkini. Alih-alih memelihara daftar yang terurut sepenuhnya seperti LRU, algoritma ini menggerakkan "jarum jam" untuk menemukan entri yang akan diganti, sambil membersihkan bit referensi di sepanjang jalan. Desain ini secara signifikan mengurangi kebutuhan akan *locking* yang berlebihan dan pembaruan yang sering, sambil tetap mempertahankan manfaat dari LRU. Hasilnya, **CLOCK** menawarkan keseimbangan praktis antara mencapai *miss ratio* yang rendah dan menjaga skalabilitas serta kesederhanaan yang menjadi ciri khas **FIFO** **chen2025lazy**.

Namun, terlepas dari keunggulannya, algoritma **CLOCK** masih mengalami masalah promosi yang tidak perlu **Eads2014NbQCLOCK, chen2025lazy**. Karena setiap akses akan mengatur bit referensi tanpa memandang frekuensi atau kepentingan akses, bahkan akses tunggal atau yang jarang terjadi dapat mencegah sebuah entri untuk digusur. Akibatnya, *cache* dapat mempertahankan item yang memberikan sedikit manfaat sambil menggusur item lain dengan potensi penggunaan kembali yang lebih tinggi. Hal ini menyebabkan inefisiensi dalam pemanfaatan *cache*.

Perkembangan terkini dalam *machine learning* telah membuka kemungkinan untuk menggunakannya sebagai komponen pengambilan keputusan tambahan untuk mengurangi promosi yang tidak perlu dalam sistem *cache*. Dengan belajar dari pola akses dan mengidentifikasi objek yang kemungkinan besar tidak akan digunakan kembali, model *machine learning* dapat membantu mencegah *cache* mempromosikan entri yang memberikan sedikit manfaat. Pendekatan ini memungkinkan mekanisme *caching* untuk membuat keputusan yang lebih selektif dan terinformasi tentang objek mana yang layak dipertahankan. Dengan demikian, *machine learning* dapat melengkapi algoritma tradisional seperti **CLOCK** dengan meningkatkan efisiensi, mengurangi *overhead* manajemen, dan beradaptasi secara dinamis terhadap karakteristik beban kerja yang berubah **Cunningham2008Supervised, Chapelle2006SemiSupervised, Hilbe2009Logistic, Hosmer2013Applied**.

1.2 Rumusan Masalah

Berdasarkan pembahasan pada Latar Belakang, masalah utama yang dibahas dalam penelitian ini adalah bagaimana mengintegrasikan teknik *machine learning* ke dalam algoritma penggantian *cache* **CLOCK** untuk mengurangi promosi yang tidak perlu dan meningkatkan efisiensi sistem secara keseluruhan. Secara spesifik, studi ini bertujuan untuk mengeksplorasi bagaimana pengambilan keputusan berbasis data dapat membantu algoritma **CLOCK** untuk lebih mendekati perilaku penggantian yang optimal. Dengan demikian, penelitian ini berupaya untuk meningkatkan

akurasi dan skalabilitas dari manajemen *cache* berbasis CLOCK.

1.3 Tujuan Penelitian

Tujuan utama dari penelitian ini adalah sebagai berikut:

1. Mengintegrasikan model *machine learning* ke dalam algoritma penggantian *cache* CLOCK untuk menyempurnakan proses pengambilan keputusannya.
2. Mengevaluasi efisiensi dan efektivitas algoritma CLOCK berbasis *machine learning* dalam mengurangi jumlah promosi dan meningkatkan kinerja *cache* secara keseluruhan.

1.4 Batasan Masalah

Studi ini berfokus pada integrasi teknik *machine learning* ke dalam algoritma penggantian *cache* CLOCK untuk meningkatkan efisiensinya. Penelitian ini terbatas pada lingkungan simulasi *cache* berbasis perangkat lunak, bukan implementasi perangkat keras. Studi ini mengevaluasi metrik kinerja seperti *miss ratio* dan tingkat promosi.

Ruang lingkup penelitian tidak mencakup pengembangan arsitektur *machine learning* baru atau perbandingan dengan sistem *caching* berbasis *reinforcement learning* Sutton 2005 Reinforcement yang canggih. Sebaliknya, penelitian ini menekankan analisis tentang bagaimana model ringan yang ada dapat meningkatkan pengambilan keputusan di CLOCK tanpa meningkatkan kompleksitas secara signifikan. Penelitian ini hanya berfokus pada promosi dan *miss ratio* serta tidak menekankan performa asli pada perangkat keras nyata. Penelitian ini juga terbatas pada *datasets* berbasis zipfian.

1.5 Sistematika Penulisan

Laporan tugas akhir ini dibagi menjadi lima bab sebagai berikut:

1. BAB I Pendahuluan

Bab ini menyajikan latar belakang studi, masalah utama yang akan dibahas, tujuan penelitian, dan ruang lingkup studi. Bab ini memberikan gambaran umum tentang motivasi dan signifikansi topik penelitian.

2. BAB II Tinjauan Pustaka

Bab ini membahas karya-karya sebelumnya dan studi terkait yang menjadi landasan penelitian ini. Bab ini menyoroti algoritma penggantian *cache* yang ada, pengembangan algoritma CLOCK, dan aplikasi *machine learning* sebelumnya dalam sistem *caching*.

3. BAB III Perancangan dan Implementasi Sistem

Bab ini menjelaskan desain dan implementasi sistem yang diusulkan. Bab ini menjelaskan arsitektur sistem secara keseluruhan, alur data, integrasi model *machine learning*, dan proses adaptasi model ke algoritma penggantian *cache* CLOCK.

4. BAB IV Pengujian dan Analisis

Bab ini menyajikan prosedur pengujian, pengaturan eksperimental, dan metrik evaluasi yang digunakan untuk mengukur kinerja sistem. Bab ini juga memberikan analisis hasil, membandingkan pendekatan yang diusulkan dengan metode dasar dalam hal efisiensi dan pengurangan promosi.

5. **BAB V Penutup**

Bab ini merangkum temuan dan kontribusi penelitian. Bab ini juga membahas kemungkinan perbaikan dan rekomendasi untuk pekerjaan di masa depan terkait manajemen *cache* berbasis *machine learning*.

BAB II

TINJAUAN PUSTAKA

Bab ini menyajikan tinjauan mendalam mengenai konsep-konsep dasar dan penelitian terdahulu yang menjadi landasan bagi studi ini. Pembahasan mencakup prinsip-prinsip fundamental *caching*, algoritma penggantian yang sudah mapan, serta berbagai upaya sebelumnya dalam meningkatkan sistem manajemen *cache*.

2.1 Penelitian Terdahulu

2.1.1 Evolusi Algoritma CLOCK

Algoritma CLOCK memiliki sejarah panjang dan fundamental dalam manajemen memori sistem operasi. Dikembangkan pertama kali pada awal tahun 1970-an oleh para peneliti di IBM, algoritma ini hadir sebagai solusi praktis untuk masalah biaya komputasi yang tinggi pada algoritma *Least Recently Used* atau LRU **Tang1976Cache**. Implementasi murni LRU memerlukan manipulasi senarai berantai atau tumpukan setiap kali sebuah halaman memori diakses. Hal ini menciptakan *overhead* yang tidak dapat diterima pada perangkat keras masa itu yang memiliki keterbatasan siklus prosesor. CLOCK mengatasi masalah ini dengan pendekatan aproksimasi yang cerdas menggunakan struktur data sirkular dan satu bit referensi perangkat keras.

Mekanisme inti dari CLOCK sering disebut sebagai kebijakan "Second Chance". Halaman-halaman memori disusun dalam daftar melingkar yang diibaratkan sebagai wajah jam. Sebuah penunjuk atau "jarum jam" menunjuk ke halaman yang paling tua secara kronologis. Ketika sistem memerlukan ruang kosong, jarum jam akan memeriksa bit referensi (*use bit*) dari halaman yang ditunjuk. Jika bit tersebut bernilai 1 maka itu menandakan halaman baru saja digunakan. Algoritma kemudian akan mengubah bit tersebut menjadi 0 dan memberikan kesempatan kedua pada halaman tersebut sebelum memajukan jarum ke halaman berikutnya. Jika jarum menemukan halaman dengan bit bernilai 0 maka halaman tersebut dianggap tidak aktif dan akan digusur (*evicted*) dari memori. Mekanisme ini sangat efisien karena hanya memerlukan operasi bit sederhana dan pergerakan penunjuk array. Sistem operasi modern seperti Linux dan varian BSD masih menggunakan turunan dari logika ini dalam manajemen memori virtual mereka karena kesederhanaan dan keandalannya.

2.1.2 CLOCK with Adaptive Replacement (CAR)

Meskipun efisien, algoritma CLOCK standar memiliki kelemahan yang sama dengan LRU yaitu ketidakmampuan untuk membedakan antara data yang baru saja diakses (*recency*) dan data yang sering diakses (*frequency*). Untuk mengatasi hal ini, Bansal dan Modha memperkenalkan **Clock with Adaptive Replacement (CAR)** pada tahun 2004 **Bansal2004CAR**. CAR adalah adaptasi dari algoritma ARC (*Adaptive Replacement Cache*) yang terkenal namun dimodifikasi agar tidak memerlukan penguncian global (*global lock*) yang mahal seperti pada LRU.

Keunggulan utama CAR terletak pada struktur data ganda yang dimilikinya. Algoritma

ini membagi cache menjadi dua daftar sirkular terpisah yaitu T_1 dan T_2 . Daftar T_1 menyimpan halaman yang baru diakses satu kali sedangkan daftar T_2 menyimpan halaman yang telah diakses setidaknya dua kali atau lebih. Hal ini secara efektif memisahkan data yang memiliki utilitas jangka pendek dari data yang memiliki utilitas jangka panjang. Selain dua daftar utama tersebut, CAR juga memelihara dua daftar bayangan atau *ghost lists* yang disebut B_1 dan B_2 . Daftar bayangan ini tidak menyimpan data halaman sebenarnya melainkan hanya menyimpan metadata atau riwayat dari halaman yang baru saja digusur dari T_1 atau T_2 .

CAR menggunakan parameter adaptif p untuk menentukan target ukuran dari daftar T_1 . Nilai p ini tidak statis melainkan berubah secara dinamis sebagai respons terhadap pola beban kerja. Jika terjadi *cache miss* namun metadata halaman ditemukan di B_1 maka sistem menganggap bahwa daftar T_1 terlalu kecil. Algoritma kemudian akan meningkatkan nilai p untuk memperbesar kapasitas T_1 . Sebaliknya jika metadata ditemukan di B_2 maka sistem akan mengurangi nilai p untuk memberikan lebih banyak ruang bagi T_2 . Mekanisme umpan balik ini memungkinkan CAR untuk menyesuaikan diri secara otomatis (*self-tuning*) tanpa memerlukan intervensi manual dari administrator sistem.

2.1.3 Clock-Pro

Pengembangan signifikan lainnya diperkenalkan oleh Jiang et al pada tahun 2005 melalui algoritma **Clock-Pro** Jiang2005CLOCKPro. Algoritma ini dirancang untuk mengatasi kelemahan mendasar dari algoritma berbasis LRU dan CLOCK yaitu resistansi yang buruk terhadap pemindaian sekuensial atau *scan resistance*. Dalam skenario di mana aplikasi membaca file besar sekali jalan, algoritma standar cenderung menggusur data penting yang sering digunakan demi data baru yang hanya dibaca sekali. Clock-Pro memecahkan masalah ini dengan mengadopsi prinsip dari algoritma LIRS (*Low Inter-reference Recency Set*) namun mengimplementasikannya dengan efisiensi struktur sirkular CLOCK.

Clock-Pro mengkategorikan halaman berdasarkan jarak penggunaan kembali atau *reuse distance*. Halaman dengan jarak penggunaan kembali yang pendek diklasifikasikan sebagai halaman *Hot* sedangkan halaman dengan jarak penggunaan panjang atau tidak diketahui diklasifikasikan sebagai halaman *Cold*. Secara fisik, Clock-Pro menyatukan semua halaman ini dalam satu daftar sirkular besar untuk efisiensi. Namun secara logis, halaman-halaman tersebut diperlakukan berbeda. Halaman *Hot* ditempatkan di kepala daftar dan diberikan perlindungan kuat dari pengusuran.

Fitur unik dari Clock-Pro adalah adanya masa percobaan atau *Test Period* untuk halaman *Cold*. Ketika halaman baru dimuat ke dalam memori, halaman tersebut tidak langsung dianggap penting. Halaman tersebut masuk dalam status *Cold* dan memulai masa percobaan. Jika halaman tersebut diakses kembali selama masa percobaan masih berlaku, halaman tersebut terbukti memiliki lokalitas temporal yang tinggi dan segera dipromosikan menjadi status *Hot*. Sebaliknya, jika halaman tersebut tidak diakses hingga masa percobaannya habis, halaman tersebut akan dengan cepat digusur dari memori. Mekanisme ini memungkinkan Clock-Pro untuk menyaring gangguan dari operasi pemindaian data besar dan mempertahankan data yang benar-benar berharga di dalam cache. Hasil eksperimen menunjukkan bahwa Clock-Pro mampu mencapai rasio *hit* yang setara dengan algoritma kompleks seperti LIRS namun dengan biaya implementasi yang rendah setara dengan CLOCK standar.

2.1.4 Varian Lain: GCLOCK dan WSClock

Selain CAR dan Clock-Pro, terdapat varian lain yang patut dicatat seperti **Generalized Clock (GCLOCK)**. Algoritma ini memperluas konsep bit referensi tunggal menjadi sebuah penghitung atau *counter*. Setiap kali halaman diakses, nilai penghitung akan bertambah. Saat jarum jam melewati halaman tersebut, nilai penghitung akan dikurangi satu per satu alih-alih langsung diganti menjadi nol. Halaman hanya akan digusur jika nilai penghitungnya mencapai nol. Pendekatan ini memberikan granularitas yang lebih halus dalam mengukur frekuensi akses halaman.

Varian lain yang sangat krusial dalam konteks memori virtual adalah **WSClock** atau *Working Set Clock*. Algoritma ini menggabungkan prinsip *Working Set* dengan mekanisme CLOCK. WSClock tidak hanya memeriksa bit referensi tetapi juga memeriksa waktu terakhir kali halaman digunakan. Halaman hanya akan tetap berada di memori jika halaman tersebut dianggap berada dalam set kerja (*working set*) aktif dari proses yang sedang berjalan. Pendekatan ini sangat efektif dalam mencegah fenomena *thrashing* pada sistem multiprogramming yang padat.

2.1.5 Offline CLOCK

Offline CLOCK bukanlah algoritma praktis yang dapat diimplementasikan pada sistem operasi nyata (*online*), melainkan sebuah tolok ukur teoretis atau *oracle algorithm*. Berbeda dengan algoritma Belady (OPT) yang mengganti halaman dengan waktu akses terjauh di masa depan tanpa mempedulikan struktur data dasar, Offline CLOCK secara spesifik mencari batas atas kinerja yang dapat dicapai oleh struktur *circular buffer* standar milik CLOCK **chen2025lazy**. Tujuannya adalah untuk mengisolasi inefisiensi yang disebabkan oleh keputusan penggantian yang salah dari inefisiensi yang disebabkan oleh batasan struktural algoritma itu sendiri.

Masalah utama yang diidentifikasi dalam analisis Offline CLOCK adalah fenomena "promosi buta". Pada algoritma CLOCK standar, setiap kali jarum jam melewati halaman dengan bit referensi bernilai 1, halaman tersebut secara otomatis diberi kesempatan kedua dan diizinkan tetap di memori. Keputusan ini diambil tanpa mengetahui apakah halaman tersebut akan benar-benar diakses lagi dalam waktu dekat. Seringkali, halaman ini hanya memenuhi *cache* ("cache pollution") sebelum akhirnya digusur pada putaran jarum berikutnya tanpa pernah diakses kembali selama periode kesempatan kedua tersebut. Promosi semacam ini dikategorikan sebagai promosi sia-sia atau *useless promotion*.

Mekanisme kerja Offline CLOCK melibatkan simulasi iteratif pada *trace* akses yang sama. Pada iterasi pertama, simulator menjalankan logika CLOCK standar dan mencatat setiap keputusan promosi yang dilakukan. Jika sebuah halaman dipromosikan (bit referensi diubah dari 1 ke 0) tetapi kemudian halaman tersebut digusur tanpa ada akses susulan (*re-reference*), maka promosi tersebut ditandai sebagai tindakan yang tidak optimal. Pada iterasi berikutnya, algoritma menggunakan pengetahuan masa depan (*future knowledge*) ini untuk memodifikasi perilaku jarum jam: saat bertemu halaman tersebut, algoritma akan "mematikan" kesempatan keduanya dan langsung menggusurnya, seolah-olah bit referensinya bernilai 0. Proses ini diulang terus-menerus hingga mencapai titik konvergensi di mana jumlah promosi sia-sia diminimalkan. Hasil eksperimen menunjukkan bahwa Offline CLOCK mampu mendekati kinerja optimal Belady, yang membuktikan bahwa ruang improvisasi terbesar untuk algoritma berbasis CLOCK terletak pada akurasi prediksi promosi, bukan pada kompleksitas struktur datanya **chen2025lazy**.

2.2 Konsep Dasar

2.2.1 Distribusi Zipfian

Distribusi Zipf adalah model statistik yang sering muncul dalam data perilaku manusia. Model ini menyatakan bahwa frekuensi suatu kejadian berbanding terbalik dengan peringkatnya. Jika kita mengurutkan halaman web dari yang paling populer hingga yang paling jarang dikunjungi maka frekuensi aksesnya akan mengikuti pola tertentu.

Secara matematis probabilitas akses (P_i) untuk objek pada peringkat ke- i dapat dituliskan sebagai berikut

$$P_i = \frac{\Omega}{i^\alpha} \quad (2.1)$$

Di sini α adalah parameter kemiringan yang menentukan seberapa tajam perbedaan antara objek populer dan tidak populer. Nilai Ω adalah konstanta normalisasi. Pola ini sangat relevan untuk sistem *cache*. Karena sebagian kecil halaman web (peringkat atas) menerima porsi lalu lintas yang sangat besar maka sistem *cache* menjadi sangat efektif. Kita hanya perlu menyimpan sejumlah kecil halaman populer tersebut untuk melayani banyak permintaan pengguna. Penelitian klasik oleh **Breslau1999Web** menyelidiki fenomena ini secara mendalam. Mereka menemukan bahwa permintaan web mengikuti distribusi "Zipf-like" dengan nilai α berkisar antara 0.6 hingga 0.8. Temuan dari **Breslau1999Web** memberikan implikasi penting mengenai ukuran *cache*.

- **Efisiensi Awal Tinggi**

Karena popularitas terkonsentrasi pada sedikit objek maka *cache* berukuran kecil pun bisa menghasilkan *Hit Ratio* yang tinggi.

- **Pertumbuhan Logaritmik**

Manfaat penambahan ukuran *cache* akan menurun seiring waktu. Rasio *hit* tumbuh secara logaritmik terhadap ukuran *cache*. Ini berarti menggandakan ukuran penyimpanan tidak akan menggandakan kinerja sistem.

Kesimpulannya adalah pola Zipf menjamin bahwa *web caching* adalah strategi yang efisien. Namun administrator sistem harus menyadari bahwa memperbesar kapasitas penyimpanan secara berlebihan akan memberikan hasil yang semakin kecil (diminishing returns).

2.2.2 Cache

Cache didefinisikan sebagai komponen perangkat keras atau perangkat lunak yang bertindak sebagai lapisan penyimpanan perantara berukuran kecil namun memiliki latensi akses yang sangat rendah. Secara arsitektural, *cache* ditempatkan di antara konsumen data (seperti prosesor atau aplikasi pengguna) dan penyedia data utama (seperti memori utama atau disk) yang memiliki kapasitas besar namun kecepatan akses yang jauh lebih lambat **Hennessy2011Computer**. Fungsi fundamental dari *cache* adalah untuk menyalin dan menyimpan sementara subset data dari sumber asli, sehingga permintaan data di masa depan dapat dilayani dengan lebih cepat. Keberadaan *cache* sangat krusial untuk menjembatani kesenjangan kinerja (*performance gap*) antara kecepatan pemrosesan CPU yang meningkat secara eksponensial dan kecepatan akses memori yang tumbuh secara linear, sebuah fenomena yang dikenal sebagai *Memory Wall* **Wulf1995Hitting**.

Efektivitas mekanisme *caching* tidak terjadi secara kebetulan, melainkan didasarkan pada prinsip fundamental ilmu komputer yang dikenal sebagai *Principle of Locality*. Prinsip ini menyatakan bahwa program komputer cenderung mengakses bagian ruang alamat yang relatif kecil pada waktu tertentu **Denning2005Locality**. Lokalitas ini terbagi menjadi dua dimensi utama. Pertama, *Temporal Locality*, yang menyiratkan bahwa jika suatu item data diakses saat ini, besar kemungkinan item yang sama akan diakses kembali dalam waktu dekat. Kedua, *Spatial Locality*, yang menyiratkan bahwa jika suatu item diakses, maka item-item yang berada di alamat memori yang berdekatan kemungkinan besar juga akan segera diakses **Tanenbaum2014Modern**. Tanpa adanya perilaku lokalitas ini pada beban kerja sistem, penggunaan *cache* tidak akan memberikan keuntungan kinerja yang signifikan dan justru dapat menambah *overhead* sistem.

Kinerja sebuah sistem *cache* umumnya diukur menggunakan metrik *hit ratio* dan *miss ratio*. *Cache hit* terjadi ketika data yang diminta oleh sistem ditemukan di dalam *cache*, yang memungkinkan data tersebut dikirimkan dengan latensi minimal. Sebaliknya, *cache miss* terjadi ketika data tidak ditemukan, sehingga sistem dipaksa untuk mengambil data dari lapisan penyimpanan di bawahnya yang lebih lambat, menimbulkan penalti waktu akses (*miss penalty*) **Jacob2007Memo**. Dalam sistem skala besar seperti *Content Delivery Network* (CDN) atau basis data terdistribusi, biaya dari sebuah *miss* bisa sangat mahal, tidak hanya dalam hal latensi tetapi juga dalam penggunaan *bandwidth* jaringan dan beban I/O pada server asal **Vakili2020Cache**. Oleh karena itu, optimasi algoritma manajemen *cache* difokuskan untuk meminimalkan *miss ratio* serendah mungkin.

Implementasi *cache* tidak terbatas pada satu lapisan saja, melainkan membentuk sebuah hierarki memori yang kompleks.

Pada tingkat perangkat keras, terdapat L1, L2, dan L3 *cache* yang tertanam langsung pada prosesor. Pada tingkat sistem operasi, terdapat *page cache* yang memanfaatkan RAM yang tidak terpakai untuk menyimpan blok data dari disk guna mempercepat operasi baca-tulis sistem berkas **Love2010Linux**. Lebih jauh lagi di tingkat aplikasi dan jaringan, terdapat *web cache* dan *proxy cache* yang menyimpan objek web statis maupun dinamis untuk mengurangi beban trafik internet global. Meskipun implementasinya beragam, tantangan utamanya tetap sama: bagaimana mengelola ruang penyimpanan yang terbatas untuk menyimpan data yang paling bernilai. Karena kapasitas *cache* jauh lebih kecil dibandingkan total data yang ada, mekanisme pengusuran atau *eviction policy* menjadi komponen kritis yang menentukan data mana yang harus dibuang ketika ruang penuh untuk mengakomodasi data baru yang masuk **Podlipnig2003A**.

2.2.3 Algoritma Eviction

Algoritma *eviction*, yang sering juga disebut sebagai *page replacement algorithm* dalam konteks sistem operasi, adalah jantung dari manajemen memori modern. Fungsi utamanya adalah mengambil keputusan kritis di saat kondisi memori penuh yaitu menentukan objek mana yang harus dikorbankan (*victim*) untuk memberi ruang bagi objek baru yang diminta oleh sistem **Belady1966A**. Keputusan ini tidak boleh diambil secara sembarangan karena biaya dari sebuah kesalahan prediksi sangatlah mahal. Jika algoritma membuang data yang sebenarnya akan segera dibutuhkan kembali, sistem akan mengalami *cache miss* yang memaksa prosesor untuk mengambil data dari penyimpanan sekunder (disk) yang kecepatannya bisa ribuan kali lebih lambat dibandingkan memori utama. Oleh karena itu, tujuan utama dari setiap algoritma *eviction* adalah meminimalkan frekuensi kejadian *page fault* atau *cache miss* dengan memprediksi

data mana yang memiliki utilitas terendah di masa depan.

Berikut adalah pembahasan mendalam mengenai algoritma-algoritma fundamental yang menjadi basis pengembangan strategi manajemen *cache*.

1. Algoritma Optimal (OPT/MIN)

Sebelum membahas algoritma praktis, penting untuk memahami batas atas teoretis yang dikenal sebagai Algoritma Belady atau OPT. Algoritma ini bekerja dengan prinsip sederhana namun mustahil dilakukan secara *real-time* yaitu "Gantilah halaman yang tidak akan digunakan untuk jangka waktu paling lama di masa depan" **Belady1966A**. Karena OPT membutuhkan pengetahuan sempurna mengenai masa depan (*future knowledge*), algoritma ini tidak dapat diimplementasikan pada sistem nyata. Namun, OPT sangat penting dalam riset sebagai tolok ukur (*baseline*) untuk mengukur efisiensi algoritma lain. Jika sebuah algoritma baru memiliki kinerja yang mendekati OPT, maka algoritma tersebut dapat dianggap sangat efisien.

2. First-In First-Out (FIFO)

FIFO adalah pendekatan yang paling naif dan mudah diimplementasikan. Sistem mengelola bingkai *cache* sebagai sebuah antrian sirkular murni. Data yang masuk paling awal dianggap sebagai data tertua dan akan menjadi kandidat utama untuk digusur, tanpa mempedulikan pola akses yang terjadi setelah data tersebut masuk. Kelemahan fatal FIFO adalah ketidakmampuannya membedakan antara data "tua" yang sudah usang dengan data "tua" yang masih sangat sering digunakan (misalnya variabel global atau kernel sistem). Selain itu, FIFO menderita anomali terkenal yang disebut *Belady's Anomaly*, sebuah fenomena kontra-intuitif di mana penambahan kapasitas memori fisik justru menyebabkan peningkatan jumlah *cache miss* pada pola akses tertentu. Hal ini membuktikan bahwa FIFO bukanlah *stack algorithm* yang stabil.

3. Least Recently Used (LRU)

LRU dikembangkan untuk mengatasi kelemahan FIFO dengan memanfaatkan prinsip *Temporal Locality*. Asumsi dasarnya adalah masa lalu merupakan cerminan masa depan halaman yang baru saja diakses kemungkinan besar akan diakses lagi segera, sedangkan halaman yang sudah lama tidak disentuh kemungkinan besar sudah tidak relevan. Oleh karena itu, LRU mengusur halaman yang waktu akses terakhirnya paling lampau. Meskipun sangat efektif dan bebas dari anomali Belady, implementasi LRU murni sangat mahal secara komputasi. Sistem harus memelihara sebuah tumpukan (*stack*) atau daftar berantai ganda (*doubly linked list*) dari seluruh halaman di memori. Setiap kali terjadi akses memori bahkan untuk sekadar membaca data sistem harus mencabut halaman tersebut dari posisinya di tengah antrian dan memindahkannya ke posisi paling depan (MRU). Operasi pembaruan pointer ini membutuhkan instruksi atomik dan penguncian (*locking*) yang dapat menghambat kinerja pada sistem *multi-core* berkecepatan tinggi. Selain itu, LRU rentan terhadap masalah *Scan Resistance*, di mana proses pembacaan data besar sekali lewat (seperti *backup* database) dapat menyapu bersih seluruh isi *cache* yang berharga, menggantinya dengan data yang hanya dipakai sekali.

4. CLOCK (Second Chance)

Algoritma CLOCK hadir sebagai solusi pragmatis untuk mendekati kinerja LRU tanpa membebani perangkat keras dengan manipulasi *linked list* yang berat. CLOCK menggunakan struktur data *circular buffer* sederhana dan memanfaatkan satu bit perangkat keras yang disebut

reference bit (atau *use bit*) yang tertanam pada setiap *Page Table Entry* (PTE). Cara kerjanya menyerupai jarum jam.

1. Sebuah penunjuk ("jarum jam") menunjuk ke halaman kandidat penggusuran saat ini.
2. Ketika memori penuh, algoritma memeriksa *reference bit* dari halaman yang ditunjuk.
3. Jika bit bernilai 1 (halaman baru saja diakses), algoritma tidak langsung menggusurnya. Sebaliknya, ia memberikan "kesempatan kedua" dengan mengubah bit tersebut menjadi 0, lalu memajukan jarum ke halaman berikutnya.
4. Jika bit bernilai 0 (halaman tidak diakses sejak pemeriksaan terakhir jarum jam), maka halaman tersebut dianggap sudah cukup tua dan aman untuk digusur.

Mekanisme ini memungkinkan halaman yang sering diakses untuk tetap bertahan di memori (karena bit-nya akan diset kembali menjadi 1 oleh CPU saat diakses), sementara halaman yang jarang diakses akan perlahan terdegradasi menjadi 0 dan akhirnya tergusur. Karena efisiensinya yang tinggi dan kebutuhan memori yang minimal, CLOCK menjadi algoritma standar *de-facto* pada banyak sistem operasi modern, termasuk varian BSD dan Linux **Linux**.

5. Least Frequently Used (LFU)

Berbeda dengan LRU yang berfokus pada *kapan* terakhir kali data diakses, LFU berfokus pada *seberapa sering* data tersebut diakses. Algoritma ini memelihara sebuah penghitung frekuensi untuk setiap objek di dalam *cache*. Objek dengan nilai penghitung terendah akan menjadi korban penggusuran saat memori penuh. Secara intuitif, LFU lebih akurat daripada LRU dalam mempertahankan data yang populer dalam jangka panjang.

Namun, LFU memiliki dua kelemahan fatal yang membuatnya jarang digunakan dalam bentuk murninya. Pertama adalah kompleksitas implementasi. Untuk menemukan objek dengan frekuensi terendah secara efisien, LFU memerlukan struktur data yang kompleks seperti *min-heap* atau *bucket queue*, yang menambah *overhead* manajemen memori secara signifikan. Kedua adalah masalah "Cache Pollution" akibat akumulasi frekuensi masa lalu. Sebuah objek yang sangat populer di masa lalu (misalnya, data inisialisasi program) akan memiliki nilai frekuensi yang sangat tinggi. Meskipun objek tersebut tidak pernah diakses lagi, ia akan tetap "bertengger" di dalam *cache* dan sulit digusur, menghalangi masuknya objek baru yang mungkin lebih relevan saat ini. Untuk mengatasi ini, varian modern seperti LFU-Aging atau Window-LFU diperkenalkan untuk menurunkan nilai frekuensi objek seiring berjalannya waktu (*decay*).

6. Segmented LRU (SLRU)

SLRU dirancang spesifik untuk mengatasi kelemahan utama LRU yaitu ketahanan yang buruk terhadap pola akses *scan* (pembacaan data sekali lewat). Algoritma ini membagi *cache* menjadi dua segmen terpisah yaitu segmen *Probationary* (percobaan) dan segmen *Protected* (terlindung). Setiap objek yang baru pertama kali diakses akan dimasukkan ke dalam segmen *Probationary*. Hanya jika objek tersebut diakses kembali (*cache hit*) saat masih berada di segmen percobaan, objek tersebut akan dipromosikan ke segmen *Protected*.

Segmen *Protected* biasanya dikelola sebagai LRU standar, namun dengan aturan khusus jika segmen ini penuh, objek terlama akan diturunkan kembali ke segmen *Probationary* (bukan

langsung dibuang). Penggusuran hanya terjadi pada ujung paling bawah dari segmen *Probationary*. Mekanisme ini menciptakan filter yang efektif. Operasi *scan* besar-besaran hanya akan membanjiri segmen *Probationary* dan kemudian segera dibuang, tanpa mengganggu objek-objek penting yang tersimpan aman di segmen *Protected*. SLRU menjadi basis dari banyak algoritma modern yang digunakan dalam sistem *storage* dan *database*.

7. Random Replacement (RR)

Meskipun terdengar sederhana dan tidak cerdas, *Random Replacement* memiliki tempat tersendiri dalam hierarki algoritma *eviction*. Sesuai namanya, algoritma ini memilih korban secara acak dari kumpulan objek yang ada saat memori penuh. Keuntungan utamanya adalah kesederhanaan ekstrem dan tidak adanya kebutuhan untuk menyimpan metadata sejarah akses (seperti *timestamp* atau *counter*). Hal ini membuat RR sangat menarik untuk implementasi perangkat keras dengan sumber daya sangat terbatas, seperti pada *instruction cache* di beberapa arsitektur prosesor ARM.

Secara statistik, kinerja RR memang lebih buruk daripada LRU pada beban kerja dengan lokalitas temporal standar. Namun, RR memiliki sifat yang sangat stabil ia tidak memiliki kasus terburuk (*worst-case scenario*) yang patologis seperti halnya LRU pada pola akses *looping* yang sedikit lebih besar dari ukuran *cache* (*thrashing*). Dalam skenario simulasi stokastik atau saat pola akses benar-benar acak tanpa lokalitas, RR sering kali memberikan kinerja yang setara dengan algoritma yang jauh lebih kompleks.

8. 2Q (Two Queue)

Algoritma 2Q dikembangkan secara spesifik untuk menutup celah kelemahan LRU yang rentan terhadap polusi *cache* akibat operasi pemindaian (*scan*). Pada LRU standar, data yang dibaca sekali lewat akan langsung masuk ke posisi "terbaru" dan mendepak data penting lainnya. 2Q mengatasi ini dengan sikap skeptis terhadap data baru: "Jangan percaya data baru sampai ia membuktikan dirinya berguna."

Sesuai namanya, 2Q menggunakan dua jenis antrian utama. Antrian pertama (*A1*) dikelola secara FIFO dan berfungsi sebagai zona penyangga untuk data yang baru pertama kali dilihat. Antrian kedua (*Am*) dikelola secara LRU dan dikhususkan untuk data yang telah terbukti "panas" (diakses setidaknya dua kali). Ketika sebuah objek pertama kali diakses, ia hanya diizinkan masuk ke *A1*. Jika objek tersebut tidak diakses lagi hingga mencapai ujung antrian *A1*, ia akan langsung dibuang. Hanya jika objek tersebut diakses kembali saat masih berada di dalam sistem, ia akan dipromosikan ke antrian *Am*. Mekanisme ini secara efektif menyaring gangguan dari operasi *scan* atau akses satu kali, menjaga antrian utama tetap bersih untuk data yang benar-benar berharga.

9. Most Recently Used (MRU)

MRU adalah algoritma yang sering disalahpahami karena logikanya yang kontrainuitif ia membuang data yang justru paling baru digunakan. Sekilas, ini bertentangan dengan prinsip *Temporal Locality* yang menjadi dasar LRU. Namun, MRU memiliki satu *use case* spesifik di mana ia mengalahkan semua algoritma lain: Pola Akses Siklis (*Cyclic Access Patterns*).

Bayangkan sebuah sistem yang harus membaca data A, B, C, dan D secara berulang-ulang (loop), tetapi kapasitas *cache* hanya cukup untuk 3 item. LRU akan gagal total (*Hit Rate 0%*) karena setiap kali akses baru terjadi, ia akan membuang data terlama yang justru akan segera dibutuhkan kembali (Fenomena *Thrashing*). Dalam skenario ini, data "terbaru" adalah data

yang paling tidak berguna karena ia baru akan dibutuhkan lagi setelah satu putaran penuh. MRU mengenali pola ini dan memilih untuk membuang data yang baru saja diproses, sehingga mempertahankan data lama yang antriannya akan segera tiba. Oleh karena itu, MRU sangat vital untuk operasi basis data seperti *Full Table Scans*.

10. Adaptive Replacement Cache (ARC)

Kelemahan terbesar dari algoritma seperti SLRU atau 2Q adalah ketergantungannya pada parameter statis (misalnya, berapa persen memori untuk segmen percobaan?). Parameter yang optimal untuk satu beban kerja bisa menjadi bencana untuk beban kerja lain. ARC memecahkan masalah ini dengan menghilangkan parameter manual sepenuhnya. Algoritma ini bersifat *self-tuning*, secara dinamis menyeimbangkan alokasi memori antara data yang baru (*Recency*) dan data yang sering (*Frequency*) berdasarkan kondisi lalu lintas data saat itu.

Kunci kejeniusan ARC terletak pada penggunaan "Daftar Hantu" (*Ghost Lists*). ARC tidak hanya melacak data yang ada di memori, tetapi juga melacak riwayat data yang baru saja dibuang. Jika terjadi *cache miss* tetapi metadata objek ditemukan di daftar hantu segmen *Recency*, ARC tahu bahwa ia seharusnya memberi ruang lebih besar untuk data baru, sehingga ia secara otomatis memperbesar segmen tersebut. Sebaliknya, jika hit terjadi di daftar hantu segmen *Frequency*, ia akan menggeser keseimbangan ke arah frekuensi. Hal ini membuat ARC sangat tangguh dan adaptif terhadap perubahan pola akses yang fluktuatif tanpa perlu campur tangan administrator.

11. Low Inter-reference Recency Set (LIRS)

LIRS mengajukan kritik fundamental terhadap LRU: "Jarak waktu sejak akses terakhir (*Recency*) bukanlah prediktor terbaik untuk akses masa depan." Sebagai gantinya, LIRS menggunakan metrik *Inter-Reference Recency* (IRR), yaitu jarak antara dua akses terakhir pada blok yang sama. Asumsinya adalah jika sebuah blok memiliki IRR yang pendek di masa lalu, ia kemungkinan besar akan memiliki IRR yang pendek lagi di masa depan.

LIRS membagi tumpukan *cache* menjadi dua set: LIR (*Low IRR*) untuk data inti yang sangat stabil, dan HIR (*High IRR*) untuk data yang jarang atau belum tentu berguna. Sebagian besar memori dialokasikan untuk LIR, sementara HIR hanya mendapatkan porsi kecil sebagai penyangga. Jika sebuah halaman di set LIR mulai jarang diakses, statusnya akan didemosi menjadi HIR. Sebaliknya, halaman HIR yang membuktikan diri dengan akses cepat akan dipromosikan. Meskipun implementasinya jauh lebih kompleks daripada LRU, LIRS terbukti secara empiris memiliki kinerja yang jauh lebih unggul dalam menangani beban kerja dengan lokalitas lemah (*weak locality*), seperti yang sering ditemukan pada server basis data modern.

12. TinyLFU / Window TinyLFU (W-TinyLFU)

Jika LFU adalah algoritma yang ideal namun terlalu mahal, TinyLFU adalah versi "paket hemat" yang cerdas. Masalah utama LFU tradisional adalah kebutuhan memori yang masif untuk menyimpan metadata frekuensi dari miliaran objek, termasuk objek yang sudah lama dibuang. TinyLFU memecahkan masalah ini dengan menggunakan struktur data probabilistik yang disebut *Count-Min Sketch*. Struktur ini mirip dengan *Bloom Filter*, namun digunakan untuk menghitung frekuensi secara aproksimasi dengan penggunaan memori yang sangat kecil.

Inovasi terbesar TinyLFU bukanlah pada cara ia membuang data, melainkan pada cara ia *menerima* data (*Admission Policy*). Sebagian besar algoritma *cache* menerima semua data baru secara membabi buta ("Cache Miss? Masukkan!"). TinyLFU bersikap elitis: "Apakah data

baru ini lebih berharga daripada korban yang akan dibuang?”. Saat memori penuh, TinyLFU membandingkan estimasi frekuensi kandidat korban dengan estimasi frekuensi data baru. Jika data baru memiliki frekuensi yang lebih rendah, data tersebut ditolak masuk (*rejected*). Hal ini mencegah data sampah mencemari *cache*. Varian W-TinyLFU (Window TinyLFU) menambahkan filter jendela LRU kecil di depan *cache* utama untuk melindungi data baru yang belum sempat membangun frekuensi (*burst protection*), menjadikannya standar industri modern pada sistem seperti Caffeine (Java) dan Ristretto (Go).

13. SIEVE

Di tengah tren algoritma yang semakin kompleks, SIEVE **Zhang2024**SIEVE hadir dengan filosofi radikal: “Kesederhanaan adalah kunci.” Ditemukan pada tahun 2023, SIEVE membuktikan bahwa struktur data FIFO sederhana bisa mengalahkan LRU yang kompleks jika dikelola dengan benar. SIEVE tidak memerlukan struktur senarai berantai ganda (*doubly linked list*) yang berat dan membutuhkan penguncian (*lock*) setiap kali data diakses. Sebaliknya, SIEVE hanya menggunakan satu antrian FIFO dan satu bit “dikunjungi” (*visited bit*) per objek.

Mekanisme kuncinya adalah “Promosi Malas” (*Lazy Promotion*). Saat terjadi *cache hit*, SIEVE hanya mengubah bit menjadi 1, tanpa memindahkan posisi objek dalam antrian. Pemindahan atau pembuangan hanya terjadi saat pemindaian eviksi. Sebuah “tangan” pemindai bergerak dari ekor ke kepala antrian. Jika tangan menemukan objek dengan bit 1, bit tersebut diubah menjadi 0 dan tangan lanjut bergerak (memberi kesempatan kedua). Jika tangan menemukan bit 0, objek tersebut dibuang. Kejeniusan SIEVE terletak pada “Demosi Cepat” (*Quick Demotion*): objek baru selalu masuk di kepala antrian, dan tangan pemindai bergerak menuju kepala. Artinya, objek baru yang tidak diakses lagi akan sangat cepat bertemu dengan tangan pemindai dan dibuang. Ini membuat SIEVE sangat efisien membuang “one-hit wonders” yang umum pada trafik web modern.

14. Greedy-Dual-Size-Frequency (GDSF)

Dalam konteks *web caching* atau CDN, tidak semua *cache miss* diciptakan setara. Mengambil ulang file teks 1KB jauh lebih cepat daripada mengambil ulang file video 1GB. Algoritma standar seperti LRU atau LFU buta terhadap ukuran dan biaya pengambilan (*latency/cost*) ini. GDSF **Cherkasova1998**GDSF menggabungkan tiga metrik sekaligus: Frekuensi, Ukuran, dan Biaya. Setiap objek diberikan nilai prioritas H menggunakan rumus:

$$H = L + \frac{\text{Frequency} \times \text{Cost}}{\text{Size}}$$

di mana L adalah faktor inflasi usia (*aging factor*).

Objek dengan nilai H terendah akan dibuang saat memori penuh. Komponen *Size* di penyebut memastikan bahwa objek kecil lebih disukai untuk disimpan (karena membuang satu objek besar bisa memberi ruang untuk banyak objek kecil, meningkatkan *Hit Rate*). Komponen *Cost* memastikan objek yang mahal untuk diambil (misalnya hasil komputasi database yang berat) diprioritaskan. Faktor L bertindak sebagai mekanisme penuaan yang mencegah objek populer lama memonopoli *cache* selamanya. GDSF adalah standar emas untuk *proxy server* di mana heterogenitas objek sangat tinggi.

15. LeCaR (Learning Cache Replacement)

Semua algoritma di atas adalah algoritma statis dengan aturan baku. Namun, beban kerja dunia nyata sering berubah-ubah secara drastis siang hari mungkin butuh LRU (Recency), malam hari butuh LFU (Frequency). LeCaR **Vietri2018LeCaR** menggunakan pendekatan *Machine Learning*, spesifiknya *Reinforcement Learning* berbasis *Regret Minimization*, untuk beradaptasi secara dinamis.

LeCaR memelihara dua "ahli" internal: satu kebijakan LRU dan satu kebijakan LFU. Saat terjadi *cache miss*, LeCaR melihat sejarah dan bertanya: "Seandainya saya mendengarkan si LRU tadi, apakah saya akan selamat (hit)? Atau seandainya saya mendengarkan si LFU?". Setiap kesalahan yang dibuat oleh satu ahli akan menambah bobot "penyesalan" (*regret*) pada ahli tersebut. Keputusan eviksi diambil secara probabilistik berdasarkan bobot kinerja terkini dari kedua ahli tersebut. Jika beban kerja berubah, bobot penyesalan akan bergeser, dan LeCaR akan secara otomatis beralih strategi dari dominasi LRU ke LFU atau sebaliknya tanpa intervensi manusia.

16. S3-FIFO (Simple, Scalable, Static FIFO)

Jika SIEVE adalah minimalis, S3-FIFO adalah strategis. Diperkenalkan pada SOSIP 2023, algoritma ini **Yang2023S3FIFO** menantang dominasi LRU dengan klaim berani: "FIFO lebih baik daripada LRU, asal punya teman." S3-FIFO menggunakan tiga antrian FIFO: *Small* (S), *Main* (M), dan *Ghost* (G).

Idenya adalah memisahkan "data turis" (akses sekali lewat) dari "data penduduk" (akses berulang). Semua data baru masuk ke antrian *Small*. Jika data tersebut keluar dari *S* tanpa pernah diakses lagi (*cache hit*), ia dianggap sampah dan dibuang (atau masuk *Ghost* untuk pelacakan). Namun, jika data tersebut diakses saat masih di *S*, ia langsung dipromosikan ke *Main*. Antrian *Main* menggunakan algoritma FIFO-Reinsertion: jika data di ujung antrian memiliki bit akses 1, ia dikembalikan ke kepala antrian (mirip CLOCK), jika 0 ia dibuang. Hasilnya adalah algoritma yang lebih sederhana dari LRU (tanpa *doubly linked list*) tetapi dengan kinerja yang sering kali mengalahkan algoritma canggih berbasis ML.

17. Hyperbolic Caching

Sebagian besar algoritma sibuk mengelola struktur data. Hyperbolic Caching memilih untuk sibuk berhitung. Algoritma ini **Blankstein2017Hyperbolic** mengkritik LRU dan LFU karena terlalu kaku. Alih-alih memelihara posisi dalam antrian, Hyperbolic Caching menghitung nilai prioritas dinamis untuk setiap objek saat eviksi diperlukan. Rumusnya sangat sederhana:

$$Priority = \frac{Frequency}{CurrentTime - LastAccessTime}$$

Objek dengan prioritas terendah akan dibuang.

Logikanya elegan: Data dengan frekuensi tinggi harus bertahan, tetapi nilai frekuensi itu harus "meluruh" (*decay*) seiring berjalannya waktu agar data lama tidak membusuk di dalam *cache*. Penyebut ($CurrentTime - LastAccessTime$) memastikan bahwa semakin lama data tidak disentuh, semakin kecil prioritasnya, tidak peduli seberapa tinggi frekuensinya di masa lalu. Ini menggabungkan dimensi *Recency* dan *Frequency* secara mulus tanpa parameter *tuning* yang rumit.

18. Multi-Queue (MQ)

Dalam sistem penyimpanan *block-level* atau database besar, satu atau dua antrian sering kali tidak cukup untuk menangkap nuansa beban kerja. MQ Zhou2001MQ menggunakan pendekatan stratifikasi sosial: membagi data ke dalam m level antrian LRU (biasanya Q_0 hingga Q_{m-1}). Data baru masuk di level terendah. Setiap kali data diakses, ia "naik kelas" ke antrian level yang lebih tinggi, yang memberinya perlindungan lebih lama (TTL lebih panjang).

Namun, MQ juga kejam. Ia memiliki mekanisme "waktu paruh" (*lifetime*). Jika sebuah objek di level tinggi tidak diakses dalam jangka waktu tertentu, ia akan ditendang turun ke level yang lebih rendah. Ini mencegah data yang dulunya panas (tapi sekarang dingin) untuk memblokir memori. MQ sangat efektif dalam membedakan data dengan frekuensi akses yang berbeda-beda secara granular, menjadikannya favorit di sistem penyimpanan *enterprise*.

2.2.4 Machine Learning

Machine learning (pembelajaran mesin) didefinisikan sebagai subbidang dari kecerdasan buatan (AI) yang berfokus pada pengembangan algoritma dan model statistik yang memungkinkan sistem komputer untuk melakukan tugas tertentu tanpa menggunakan instruksi eksplisit, melainkan mengandalkan pola dan inferensi. Secara fundamental, *machine learning* merepresentasikan pergeseran paradigma dari pemrograman konvensional. Dalam pemrograman tradisional, manusia menulis aturan logis (*rules*) dan data untuk menghasilkan jawaban. Sebaliknya, dalam *machine learning*, sistem menerima data dan jawaban (pada tipe *supervised*), lalu menghasilkan aturan atau logika keputusan itu sendiri.

Inti dari *machine learning* adalah kemampuan generalisasi. Sistem tidak sekadar menghafal data latih (*training data*) yang diberikan, melainkan mengekstraksi fungsi matematis atau struktur laten yang mendasari data tersebut. Tujuannya adalah agar model yang dihasilkan mampu beradaptasi dan memberikan prediksi yang akurat pada data baru yang belum pernah dilihat sebelumnya (*unseen data*). Proses ini seringkali dirumuskan sebagai upaya untuk meningkatkan kinerja P pada tugas T berdasarkan pengalaman E . Semakin banyak data (pengalaman) yang diberikan, semakin akurat kinerja model dalam menyelesaikan tugasnya.

Secara umum, algoritma *machine learning* diklasifikasikan ke dalam tiga kategori utama berdasarkan jenis umpan balik (*feedback*) yang tersedia selama proses pembelajaran Chapelle2006SemiSupervised, Sutton2005Reinforcement, Cunningham2008Supervised:

1. **Supervised Learning (Pembelajaran Terarah):** Model dilatih pada dataset yang memiliki label atau target yang jelas.
2. **Unsupervised Learning (Pembelajaran Tak Terarah):** Model mencari pola tersembunyi atau struktur intrinsik dari data yang tidak berlabel (contoh: *clustering*).
3. **Reinforcement Learning (Pembelajaran Penguatan):** Agen belajar mengambil serangkaian keputusan melalui interaksi dengan lingkungan berdasarkan sistem *reward* dan *punishment*.

2.2.5 Supervised Learning

Supervised learning merupakan paradigma yang paling dominan dan matang dalam aplikasi praktis *machine learning* saat ini. Dalam pendekatan ini, proses pelatihan model dianalogikan seperti seorang murid yang belajar di bawah pengawasan guru. Guru menyediakan

”kunci jawaban” (label) untuk setiap soal (fitur input) yang diberikan. Secara formal, diberikan himpunan data pelatihan $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$, di mana x_i adalah vektor fitur input dan y_i adalah variabel target yang sesuai. Tujuan algoritma adalah mempelajari fungsi pemetaan $f : X \rightarrow Y$ sedemikian rupa sehingga $f(x)$ dapat memprediksi nilai y dengan tingkat kesalahan yang seminimal mungkin pada data baru **Cunningham2008Supervised**.

Proses pembelajaran pada *supervised learning* melibatkan optimalisasi parameter model untuk meminimalkan fungsi tujuan (*objective function*) atau fungsi kerugian (*loss function*). Fungsi kerugian ini mengukur diskrepansi atau selisih antara prediksi model ($\hat{y}$) dan nilai sebenarnya (y). Melalui algoritma optimasi seperti *Gradient Descent*, model secara iteratif memperbarui bobot internalnya untuk mengurangi nilai kerugian tersebut hingga mencapai titik konvergen.

Berdasarkan tipe variabel output (y), masalah dalam *supervised learning* dapat dibagi menjadi dua kategori utama:

Klasifikasi (Classification) Terjadi ketika variabel output bersifat kategorikal atau diskrit. Model bertugas untuk memisahkan input ke dalam kelas-kelas yang berbeda. Contoh kasus meliputi deteksi *spam* (spam/bukan spam), pengenalan digit tulisan tangan (0-9), atau dalam konteks penelitian ini, klasifikasi objek *cache* menjadi *wasted* atau *not wasted*. Algoritma populer untuk klasifikasi meliputi *Logistic Regression*, *Decision Trees*, *Support Vector Machines (SVM)*, dan *Neural Networks*.

Regresi (Regression) Terjadi ketika variabel output bersifat kontinu atau numerik. Model bertugas memprediksi nilai kuantitas tertentu. Contoh penerapannya adalah prediksi harga rumah, estimasi suhu, atau peramalan beban trafik jaringan. Algoritma yang sering digunakan antara lain *Linear Regression* dan *Polynomial Regression*.

Tantangan utama dalam *supervised learning* adalah menjaga keseimbangan antara *bias* dan *variance*. Model yang terlalu sederhana mungkin mengalami *underfitting* (gagal menangkap pola data), sedangkan model yang terlalu kompleks berisiko mengalami *overfitting* (menghafal *noise* pada data latih sehingga gagal pada data uji). Teknik regularisasi dan validasi silang (*cross-validation*) sering diterapkan untuk mengatasi masalah ini dan memastikan model memiliki daya generalisasi yang baik.

2.2.6 TensorFlow

TensorFlow adalah pustaka perangkat lunak sumber terbuka yang dikembangkan oleh Tim Google Brain **tensorflow2015-whitepaper**. Awalnya dirilis pada tahun 2015. TensorFlow dirancang untuk komputasi numerik berkinerja tinggi. Pustaka ini menjadi standar industri dalam pengembangan model *machine learning* dan *deep learning*. Arsitektur TensorFlow memungkinkan penyebaran komputasi pada berbagai platform. Mulai dari CPU desktop, GPU (*Graphics Processing Unit*). Hingga TPU (*Tensor Processing Unit*) dan perangkat seluler.

Nama TensorFlow berasal dari operasi utamanya pada array multidimensi. Array ini disebut sebagai *tensors*. Dalam arsitektur ini, Aliran data direpresentasikan sebagai grafik komputasi berarah (*data flow graphs*). Setiap simpul (*node*) dalam grafik mewakili operasi matematika. Sedangkan setiap sisi (*edge*) mewakili array data multidimensi yang mengalir di antara operasi tersebut. Pendekatan ini memberikan fleksibilitas tinggi bagi pengembang. Mereka dapat mendeskripsikan transformasi data yang kompleks. Tanpa harus menangani detail alokasi memori atau sinkronisasi *thread* secara manual.

Evolusi TensorFlow dari versi 1.x ke 2.x membawa perubahan paradigma yang signifikan. Pada versi awal, TensorFlow menggunakan mode eksekusi grafik statis. Pengguna harus mendefinisikan seluruh grafik komputasi sebelum menjalankannya dalam sebuah sesi. Meskipun efisien secara komputasi, Pendekatan ini menyulitkan proses *debugging*. TensorFlow 2.0 memperkenalkan *Eager Execution* sebagai standar. Ini memungkinkan operasi dieksekusi secara imperatif. Hasil operasi dapat langsung dievaluasi baris demi baris seperti pada kode Python standar. Perubahan ini secara drastis menurunkan kurva pembelajaran bagi peneliti baru. Serta mempermudah integrasi dengan pustaka Python lain seperti NumPy.

Dalam konteks penelitian ini, TensorFlow berfungsi sebagai mesin komputasi utama (*back-end engine*). Ia menangani operasi aljabar linear intensif. Seperti perkalian matriks dan perhitungan gradien otomatis (*autodiff*) selama proses pelatihan model ML-CLOCK. Kemampuan TensorFlow untuk mengeksport model ke format standar seperti SavedModel atau HDF5. Serta konversinya ke format ONNX. Sangat krusial untuk tahap penyebaran model ke dalam simulator berbasis C++.

2.2.7 Keras

Keras adalah antarmuka pemrograman aplikasi (API) tingkat tinggi untuk jaringan saraf tiruan **chollet2015keras**. Ditulis dalam Python dan dikembangkan oleh François Chollet. Keras dirilis pertama kali pada tahun 2015. Filosofi desain Keras mengutamakan pengalaman pengembang (*developer experience*). Ia menekankan pada kemudahan penggunaan. Modularitas. Dan ekstensibilitas. Keras tidak melakukan operasi komputasi tingkat rendah secara langsung. Sebaliknya, Ia bertindak sebagai lapisan abstraksi (*wrapper*) yang berjalan di atas pustaka backend yang lebih rendah. Seperti TensorFlow. CNTK. Atau Theano.

Sejak rilis TensorFlow 2.0, Keras telah terintegrasi secara ketat sebagai modul resmi `tf.keras`. Ini menjadikan Keras sebagai gerbang utama untuk membangun model di ekosistem TensorFlow. Keras menyediakan blok bangunan fundamental untuk *deep learning*. Termasuk lapisan (*layers*). Fungsi aktivasi. Pengoptimal (*optimizers*). Dan fungsi kerugian (*loss functions*). Pengguna dapat menyusun model menggunakan dua paradigma utama. Yaitu Sequential API dan Functional API.

Sequential API adalah pendekatan yang paling sederhana. Model dibangun sebagai tumpukan lapisan linear. Cocok untuk model standar yang memiliki satu input dan satu output. Contohnya adalah *Multi-Layer Perceptron* (MLP) sederhana. **Functional API** menawarkan fleksibilitas lebih tinggi. Ia memungkinkan pembuatan topologi model yang kompleks. Seperti model dengan banyak input atau output. Lapisan berbagi (*shared layers*). Dan koneksi sisa (*residual connections*).

Dalam penelitian ini, Keras digunakan untuk mendefinisikan arsitektur *feed-forward neural network* ringan. Kesederhanaan sintaksis Keras memungkinkan iterasi cepat dalam bereksperimen dengan berbagai konfigurasi *hyperparameter*. Seperti jumlah *hidden layer*. Jumlah neuron per layer. Dan laju pembelajaran (*learning rate*). Kode berikut menunjukkan contoh definisi model MLP menggunakan Keras untuk masalah klasifikasi biner pada manajemen *cache*.

```
model = tf.keras.Sequential([
    tf.keras.layers.Dense(32, activation='relu', input_shape=(input_dim,)),
    tf.keras.layers.Dense(16, activation='relu'),
    tf.keras.layers.Dense(1, activation='sigmoid')
])
```

Penggunaan fungsi aktivasi Sigmoid pada lapisan keluaran memastikan hasil prediksi berada dalam rentang probabilitas 0 hingga 1. Yang merepresentasikan kemungkinan sebuah objek akan diakses kembali (*reused*) atau tidak (*wasted*).

2.2.8 Hubungan TensorFlow dan Keras

Sinergi antara TensorFlow dan Keras menciptakan ekosistem yang kuat untuk pengembangan AI. Keras menyediakan antarmuka yang intuitif dan ekspresif. Memungkinkan peneliti untuk fokus pada logika model daripada detail implementasi teknis. Sementara itu, TensorFlow menyediakan infrastruktur eksekusi yang tangguh dan skalabel di belakang layar. Kombinasi ini sangat relevan untuk penelitian sistem operasi. Di mana model harus dilatih dengan cepat menggunakan data *trace* yang besar. Namun tetap harus diekspor menjadi format yang sangat efisien untuk inferensi waktu nyata.

2.2.9 ONNX (Open Neural Network Exchange)

ONNX (*Open Neural Network Exchange*) adalah format standar terbuka yang dirancang untuk merepresentasikan model *machine learning* dan *deep learning* dalam struktur yang agnostik terhadap kerangka kerja (*framework*) pengembangan **Lin2019ONNC**, **Tian2020Compiling**. Standar ini mendefinisikan skema grafik komputasi yang dapat diperluas, serta spesifikasi operator bawaan dan tipe data standar. Tujuan utama dari ONNX adalah memecahkan masalah fragmentasi ekosistem AI dengan menyediakan jembatan interoperabilitas antara berbagai *framework* pelatihan seperti PyTorch, TensorFlow, dan Scikit-learn, dengan mesin inferensi (*inference engine*) seperti ONNX Runtime, TensorRT, atau OpenVINO **Jajal2024Interoperability**, **Sever2021A**.

Secara teknis, ONNX menggunakan format serialisasi Protocol Buffers (protobuf) untuk menyimpan definisi graf model. Sebuah model ONNX terdiri dari graf asiklik terarah (*Directed Acyclic Graph* atau DAG) di mana setiap simpul (*node*) merepresentasikan operasi matematika (seperti konvolusi, perkalian matriks, atau fungsi aktivasi) dan setiap sisi (*edge*) merepresentasikan aliran data dalam bentuk tensor. Struktur ini memungkinkan representasi model yang sangat portabel karena tidak bergantung pada implementasi kode sumber asli dari *framework* pembuatnya.

Komponen fundamental dari spesifikasi ONNX meliputi:

- **Definisi Graf Komputasi:** Spesifikasi yang mengatur bagaimana node operator, input, output, dan inisialisasi parameter disusun untuk membentuk aliran eksekusi model.
- **Operator Set (Opset):** Kumpulan operasi standar yang didukung. ONNX menggunakan sistem versi *opset* yang ketat untuk menjamin kompatibilitas ke belakang. Setiap rilis ONNX membawa pembaruan pada definisi operator untuk mendukung arsitektur jaringan saraf tiruan yang lebih baru dan kompleks.
- **Tipe Data Standar:** Definisi tipe data tensor yang konsisten (seperti Float32, Int8, Float16) untuk memastikan presisi numerik yang seragam di berbagai perangkat keras.

Pentingnya ONNX terletak pada kemampuannya untuk memisahkan tahap pengembangan model dari tahap penyebaran (*deployment*). Tanpa ONNX, pengembang sering kali terikat pada *stack* teknologi tertentu dari awal hingga akhir, yang mungkin tidak optimal untuk lingkungan produksi. Dengan menggunakan ONNX, model yang dilatih menggunakan PyTorch yang

dinamis dan fleksibel dapat diekspor menjadi format statis yang dioptimalkan, kemudian dijalankan di lingkungan produksi menggunakan C++ atau pada perangkat *edge* dengan sumber daya terbatas **Sever2021A**. Namun, konversi ke ONNX tidak selalu berjalan mulus; tantangan sering muncul ketika model menggunakan operasi kustom atau fitur dinamis yang belum didukung oleh versi *opset* standar saat ini.

2.2.10 ONNX Runtime

ONNX Runtime (ORT) adalah mesin inferensi lintas platform yang berkinerja tinggi, yang dikembangkan oleh Microsoft untuk mempercepat eksekusi model yang disimpan dalam format ONNX **Sever2021A**, **Lin2019ONNC**. Berbeda dengan *framework* pelatihan yang mengutamakan fleksibilitas dan kemudahan *debugging*, ONNX Runtime dirancang secara spesifik untuk meminimalkan latensi dan memaksimalkan *throughput* selama fase inferensi. ORT mencapai kinerja tinggi melalui kombinasi optimasi graf yang agresif dan arsitektur modular yang disebut *Execution Providers* (EP).

Arsitektur ONNX Runtime menerapkan strategi optimasi bertingkat yang dijalankan sebelum inferensi dimulai. Tingkatan optimasi ini mencakup:

- **Optimasi Level 1 (Dasar):** Meliputi teknik *constant folding* (menghitung nilai konstan saat kompilasi), penghapusan node redundan (seperti operasi identitas atau *dropout* yang tidak diperlukan saat inferensi), dan penyederhanaan aljabar dasar.
- **Optimasi Level 2 (Lanjutan):** Fokus pada fusi node (*node fusion*), di mana beberapa operasi kecil digabungkan menjadi satu kernel operasi besar untuk mengurangi *overhead* akses memori dan pemanggilan kernel. Contoh umum adalah fusi Conv+BN+Relu atau MatMul+Add.
- **Optimasi Level 3 (Layout):** Melibatkan transformasi tata letak memori data (misalnya konversi antara NCHW dan NHWC) untuk menyesuaikan dengan preferensi perangkat keras target agar akses memori lebih efisien.

Fitur paling krusial dari ONNX Runtime adalah mekanisme *Execution Providers* (EP). EP bertindak sebagai lapisan abstraksi yang memungkinkan ORT untuk mendelegasikan komputasi intensif ke akselerator perangkat keras spesifik tanpa mengubah kode aplikasi. Saat sebuah model dimuat, ORT akan mempartisi graf komputasi menjadi beberapa sub-graf. Sub-graf yang didukung oleh akselerator yang tersedia (seperti GPU NVIDIA melalui CUDA EP atau TensorRT EP, GPU AMD melalui ROCm EP, atau CPU Intel melalui OpenVINO EP) akan dieksekusi oleh perangkat keras tersebut **Tian2020Compiling**. Operasi yang tidak didukung oleh akselerator akan secara otomatis dialihkan (*fallback*) ke implementasi CPU standar, menjamin bahwa model tetap dapat berjalan meskipun akselerator tidak mendukung seluruh operator dalam model tersebut.

Selain itu, ONNX Runtime mendukung teknik kuantisasi lanjutan yang memungkinkan model dijalankan dengan presisi rendah (INT8 atau FP16) daripada presisi penuh (FP32). Kuantisasi ini secara signifikan mengurangi ukuran model dan penggunaan memori, serta meningkatkan kecepatan inferensi pada perangkat keras modern yang memiliki instruksi khusus untuk aritmatika presisi rendah, seperti Tensor Cores pada GPU NVIDIA atau instruksi VNNI pada CPU modern. Dengan demikian, ONNX Runtime memberikan solusi komprehensif untuk penyebaran model AI yang efisien di berbagai ekosistem, mulai dari server *cloud* berskala besar hingga perangkat IoT berdaya rendah **Jajal2024Interoperability**.

2.2.11 Trace Replay

Trace replay merupakan metodologi standar *de facto* dalam evaluasi kinerja sistem penyimpanan dan memori **Yang2021A**. Berbeda dengan simulasi berbasis eksekusi (*execution-driven simulation*) yang menjalankan instruksi program secara penuh, *trace replay* mengisolasi pola akses data dari logika komputasi aplikasi. Pendekatan ini bekerja dengan merekam urutan operasi I/O atau referensi memori dari sistem produksi ke dalam berkas log yang disebut *trace*. Berkas ini kemudian diputar ulang pada simulator yang memodelkan perilaku komponen target, seperti hierarki *cache* atau pengontrol disk.

Proses *trace replay* terdiri dari dua fase utama. Fase pertama adalah akuisisi data (*trace collection*). Pada tahap ini, alat instrumentasi seperti Valgrind, Intel Pin, atau modul kernel Linux (misalnya blktrace) disisipkan ke dalam sistem untuk menangkap setiap permintaan akses. Data yang dicatat biasanya mencakup waktu akses, jenis operasi (baca atau tulis), alamat memori atau LBA (*Logical Block Address*), dan ukuran data. Fase kedua adalah simulasi deterministik. Simulator membaca *trace* baris demi baris dan memperbarui struktur data internal yang merepresentasikan keadaan sistem, seperti senarai berantai untuk LRU atau pohon pencarian untuk indeks *cache*, tanpa membebani CPU dengan eksekusi kode aplikasi asli.

Keuntungan utama dari metode ini adalah reproduktifitas. Karena *trace* bersifat statis, peneliti dapat membandingkan berbagai algoritma penggantian *cache* (misalnya LRU vs ARC) pada beban kerja yang identik secara bit-per-bit. Hal ini menghilangkan variabilitas yang disebabkan oleh penjadwalan proses atau gangguan sistem operasi yang sering terjadi pada pengujian langsung. Selain itu, *trace replay* jauh lebih cepat daripada simulasi penuh karena mengabaikan detail instruksi non-memori.

Namun, metode ini memiliki keterbatasan inheren. *Trace replay* mengasumsikan bahwa waktu antar-permintaan bersifat independen terhadap kinerja sistem penyimpanan, padahal dalam kenyataannya, sistem yang lambat akan menunda permintaan berikutnya. Fenomena ini menyebabkan ketidakakuratan waktu (*timing skew*) pada sistem dengan antrean I/O yang dalam. Selain itu, ukuran berkas *trace* bisa menjadi sangat besar (mencapai Terabyte) untuk aplikasi yang berjalan lama, sehingga memerlukan teknik kompresi atau pengambilan sampel (*sampling*) yang cerdas untuk menjaga efisiensi penyimpanan tanpa mengorbankan akurasi statistik **Yang2021A**.

2.2.12 LibCacheSim

LibCacheSim adalah pustaka simulasi *cache* berkinerja tinggi yang dirancang untuk mengatasi keterbatasan simulator ad-hoc berbasis Python yang lambat **Yang2021A**. Dikembangkan menggunakan bahasa C dan C++, LibCacheSim mampu memproses jutaan operasi per detik, menjadikannya solusi ideal untuk mengevaluasi beban kerja skala besar seperti *trace* dari CDN (*Content Delivery Network*) atau pusat data *cloud*.

Arsitektur LibCacheSim bersifat modular dan terdiri dari beberapa komponen inti. Pertama, **Trace Reader** yang mendukung berbagai format input, mulai dari format teks sederhana (CSV), format biner LFU, hingga format standar industri seperti MSR Cambridge atau SNIA. Kedua, **Cache Model** yang merepresentasikan struktur penyimpanan data. LibCacheSim mendukung objek dengan ukuran variabel (*variable-sized objects*), fitur yang krusial untuk simulasi *web cache* di mana ukuran gambar atau video sangat bervariasi, berbeda dengan *CPU cache* yang biasanya memiliki ukuran blok tetap (*fixed cache lines*).

Kekuatan utama LibCacheSim terletak pada koleksi algoritma eviksinya. Pustaka ini tidak

hanya mengimplementasikan algoritma klasik seperti LRU (*Least Recently Used*), FIFO (*First-In First-Out*), dan LFU (*Least Frequently Used*), tetapi juga algoritma mutakhir yang adaptif terhadap pola akses kompleks. Contohnya termasuk:

- **ARC (Adaptive Replacement Cache):** Algoritma yang secara dinamis menyeimbangkan antara resensi (kebaruan) dan frekuensi dengan memelihara dua senarai terpisah untuk halaman yang baru diakses dan halaman yang sering diakses.
- **LeCaR (Learning Cache Replacement):** Pendekatan berbasis pembelajaran penguatan (*reinforcement learning*) yang menggunakan regret minimization untuk memilih antara kebijakan LRU dan LFU berdasarkan kinerja masa lalu.
- **TinyLFU:** Mekanisme penerimaan (*admission policy*) yang menggunakan struktur data probabilistik *Count-Min Sketch* untuk memfilter objek yang jarang diakses sebelum masuk ke *cache* utama, sehingga mencegah polusi *cache*.

Selain itu, LibCacheSim menyediakan antarmuka pengikatan (*binding*) untuk bahasa tingkat tinggi seperti Python, memungkinkan peneliti untuk membuat prototipe algoritma baru dengan cepat sambil tetap memanfaatkan kinerja backend C++ untuk eksekusi berat. Fitur analisis bawaannya juga memungkinkan pengguna untuk menghasilkan kurva rasio miss (*Miss Ratio Curves* - MRC) secara efisien menggunakan algoritma *stack distance* atau pendekatan sampling seperti SHARDS, yang sangat berguna untuk menentukan ukuran *cache* yang optimal untuk beban kerja tertentu.

2.2.13 Klasifikasi Kegagalan Cache (The Four Cs)

Untuk memahami efektivitas suatu algoritma penggantian, penting untuk membedah akar penyebab terjadinya *cache miss*. Hill (1987) mengklasifikasikan jenis kegagalan *cache* ke dalam tiga kategori utama yang dikenal sebagai "The 3Cs", yang kemudian diperluas menjadi empat kategori dalam konteks sistem multiprosesor.

1. Compulsory Miss (Cold Start Miss)

Kegagalan ini bersifat mandatori dan terjadi pada akses pertama terhadap suatu blok data. Pada saat sistem baru dinyalakan atau saat sebuah program baru dijalankan, *cache* berada dalam keadaan kosong (*cold*). Tidak ada algoritma penggantian, secerdas apa pun, yang dapat menghindari jenis kegagalan ini karena data memang belum pernah dimuat sebelumnya. Satu-satunya mitigasi yang mungkin adalah dengan meningkatkan ukuran blok (*prefetching*) untuk memanfaatkan lokalitas spasial, namun ini tidak menghilangkan *miss* pertama itu sendiri.

2. Capacity Miss

Kegagalan ini terjadi murni karena keterbatasan ruang penyimpanan. Meskipun *cache* terisi penuh dengan data yang paling sering digunakan, kapasitasnya tetap tidak cukup untuk menampung seluruh *working set* dari aplikasi yang sedang berjalan. Data yang masih dibutuhkan terpaksa dibuang untuk memberi ruang bagi data lain, dan ketika data tersebut diminta kembali, terjadi *miss*. Algoritma seperti LRU atau CLOCK berjuang keras di area ini untuk meminimalkan dampak dari kapasitas yang terbatas.

3. Conflict Miss (Collision Miss)

Jenis ini spesifik terjadi pada *cache* dengan struktur *direct-mapped* atau *set-associative*. *Miss* terjadi ketika terlalu banyak blok data berebut untuk menempati set (*bucket*) yang sama, meskipun set lain dalam *cache* masih kosong. Dalam konteks *software cache* (seperti pada sistem operasi atau aplikasi web) yang umumnya menggunakan struktur *fully associative* (menggunakan *hash map*), jenis kegagalan ini jarang menjadi masalah utama dibandingkan dengan *Capacity Miss*.

4. Coherence Miss

Pada sistem terdistribusi atau multiprosesor, *miss* dapat terjadi karena mekanisme koherensi data. Jika satu prosesor memperbarui nilai data di *cache* lokalnya, salinan data yang sama di *cache* prosesor lain harus divalidasi atau dibuang (*invalidated*) untuk menjaga konsistensi. Akses berikutnya ke data yang telah divalidasi tersebut akan menyebabkan *miss*, meskipun data tersebut secara teknis masih "ada" tetapi sudah usang (*stale*).

2.2.14 Metrik Evaluasi Kinerja

Mengevaluasi kinerja algoritma *cache* tidak cukup hanya dengan satu angka. Berikut adalah metrik-metrik kritis yang digunakan:

1. Hit Ratio dan Miss Ratio

Ini adalah metrik paling standar. *Hit Ratio* (h) didefinisikan sebagai persentase permintaan yang berhasil dilayani oleh *cache*.

$$h = \frac{\text{Jumlah Hit}}{\text{Total Permintaan}}$$

Sebaliknya, *Miss Ratio* ($m = 1 - h$) mengukur kegagalan. Meskipun populer, metrik ini sering kali menipu pada beban kerja dengan ukuran objek yang bervariasi (*variable-sized objects*).

2. Byte Hit Ratio (BHR)

Dalam konteks *web caching* atau CDN di mana ukuran objek sangat heterogen (mulai dari teks 1KB hingga video 1GB), *Hit Ratio* bisa menyesatkan. Algoritma yang hanya menyimpan objek-objek kecil akan memiliki *Hit Ratio* tinggi, tetapi *bandwidth* jaringan tetap terbebani oleh pengambilan objek besar. BHR mengukur persentase volume data (dalam *bytes*) yang dilayani langsung dari *cache*, memberikan gambaran lebih akurat mengenai penghematan *bandwidth*.

3. Average Memory Access Time (AMAT)

AMAT menggabungkan waktu akses *cache* dan penalti memori ke dalam satu metrik waktu:

$$AMAT = T_{hit} + (m \times T_{penalty})$$

Di mana T_{hit} adalah latensi akses *cache* dan $T_{penalty}$ adalah waktu yang dibutuhkan untuk mengambil data dari disk atau server asal. Dalam implementasi algoritma berbasis *Machine Learning*, T_{hit} menjadi sangat krusial karena kompleksitas inferensi model dapat meningkatkan waktu akses dasar ini, yang berpotensi meniadakan keuntungan dari penurunan m .

2.2.15 Fenomena Patologis dalam Caching

Terdapat beberapa anomali dan skenario terburuk yang harus ditangani oleh algoritma modern:

1. Belady's Anomaly

Sebuah fenomena kontra-intuitif yang ditemukan pada algoritma FIFO, di mana menambah ukuran fisik *cache* justru menyebabkan jumlah *page fault* meningkat. Hal ini membuktikan bahwa FIFO bukanlah *stack algorithm* (algoritma yang menjamin bahwa himpunan halaman pada *cache* ukuran k selalu merupakan subset dari *cache* ukuran $k + 1$). Algoritma seperti LRU dan CLOCK bebas dari anomali ini.

2. Scan Resistance

Kelemahan fatal dari LRU. Ketika sistem melakukan pembacaan sekuensial pada data yang ukurannya sedikit lebih besar dari kapasitas *cache* (misalnya *full table scan* pada database), seluruh isi *cache* yang berharga (data "panas") akan tergusur oleh data "dingin" yang hanya dibaca sekali lewat. Setelah proses *scan* selesai, *cache* terisi penuh oleh sampah, menyebabkan penurunan kinerja drastis. Algoritma modern seperti ARC, LIRS, dan 2Q dirancang khusus untuk memiliki sifat *Scan Resistant*.

3. Cache Thrashing

Kondisi di mana sistem menghabiskan lebih banyak waktu untuk mengganti halaman (keluar-masuk data) daripada melakukan komputasi nyata. Ini terjadi ketika *working set* aktif dari program jauh lebih besar daripada kapasitas *cache*. Dalam kondisi ini, algoritma penggantian terus-menerus membuang halaman yang akan segera dibutuhkan kembali.

2.3 Pendekatan Berbasis Machine Learning

Dalam dekade terakhir, penerapan teknik pembelajaran mesin (ML) untuk manajemen *cache* telah menjadi area penelitian yang sangat aktif. Premis dasarnya adalah menggantikan aturan heuristik statis (seperti LRU) dengan model yang dipelajari dari data (*data-driven policy*).

2.3.1 Formulasi Masalah sebagai Klasifikasi

Masalah penggantian *cache* dapat diformulasikan sebagai masalah klasifikasi biner. Diberikan keadaan sistem saat ini dan karakteristik sebuah objek, model harus memprediksi:

- **Kelas 0 (Evict/Reject):** Objek tidak akan diakses lagi dalam waktu dekat (atau sama sekali).
- **Kelas 1 (Keep/Promote):** Objek akan segera diakses kembali.

Algoritma seperti Hawkeye dan Glider menggunakan pendekatan ini dengan mencoba meniru keputusan algoritma optimal (Belady's OPT). Mereka melatih model (biasanya Perceptron atau SVM) menggunakan *trace* masa lalu di mana keputusan optimal sudah diketahui, kemudian menggunakan model tersebut untuk memprediksi keputusan di masa depan.

2.3.2 Learning from Belady (Imitation Learning)

Algoritma Belady memberikan batas atas kinerja teoretis dengan membuang halaman yang waktu akses berikutnya paling jauh. Meskipun tidak bisa diimplementasikan secara *online*, keputusannya dapat digunakan sebagai label (*ground truth*) untuk pelatihan *supervised learning*.

Metode seperti *Glider* menggunakan arsitektur yang terdiri dari:

1. **PC-based Predictor:** Menggunakan alamat instruksi (*Program Counter*) yang memicu akses memori sebagai fitur utama. Asumsinya adalah instruksi tertentu dalam kode program cenderung memiliki pola akses memori yang konsisten (misalnya, instruksi di dalam *loop* vs instruksi inisialisasi).
2. **SVM Classifier:** Menggunakan sejarah akses PC untuk mengklasifikasikan apakah suatu akses bersifat "cache-friendly" atau "cache-averse".

2.3.3 Reinforcement Learning (RL)

Pendekatan RL memandang manajemen *cache* sebagai masalah pengambilan keputusan sekuensial.

- **Agent:** Pengelola *cache*.
- **Environment:** Aliran permintaan data.
- **State:** Representasi isi *cache* dan sejarah akses terkini.
- **Action:** Memilih objek mana yang akan dibuang.
- **Reward:** +1 jika terjadi *hit*, -1 (atau penalti lain) jika terjadi *miss*.

Metode seperti *LeCaR* (Learning Cache Replacement) menggunakan konsep *Regret Minimization*, yang merupakan bentuk sederhana dari RL. *LeCaR* memelihara dua kebijakan dasar (biasanya LRU dan LFU) sebagai "ahli" (*experts*). Setiap kali terjadi *miss*, *LeCaR* mengevaluasi mana di antara kedua ahli tersebut yang akan memberikan hasil lebih baik jika diikuti, dan memperbarui bobot kepercayaan (*weight*) terhadap masing-masing ahli. Kebijakan akhir diambil secara probabilistik berdasarkan bobot ini.

2.3.4 Fitur Machine Learning untuk Prediksi Cache

Dalam paradigma *learning-based caching*, pemilihan fitur input (input features) sangat menentukan akurasi prediksi model. Berikut adalah fitur-fitur yang umum diekstraksi:

1. Fitur Berbasis Recency

- **Delta Time:** Selisih waktu antara waktu saat ini dengan waktu akses terakhir.
- **Stack Distance:** Posisi logis sebuah objek dalam tumpukan LRU. Nilai yang kecil menunjukkan objek sangat baru.

2. Fitur Berbasis Frequency

- **Access Count:** Jumlah total akses ke objek tersebut sejak pertama kali dimuat.
- **Frequency Density:** Frekuensi akses dibagi dengan umur objek di dalam *cache*. Ini membantu menormalisasi nilai frekuensi agar objek baru dengan frekuensi rendah tidak langsung dianggap tidak berharga dibandingkan objek lama.
- **Exponential Moving Average (EMA) Frequency:** Memberikan bobot lebih pada akses-akses terbaru, memungkinkan deteksi perubahan tren popularitas.

3. Fitur Semantik dan Atribut

- **Object Size:** Ukuran data. Algoritma seperti GDSF memprioritaskan objek kecil untuk memaksimalkan Hit Ratio, atau objek besar untuk meminimalkan trafik jaringan (Byte Hit Ratio).
- **Object Type:** Ekstensi file (misalnya .jpg vs .html) yang dapat mengindikasikan pola akses. Gambar mungkin diakses sekali untuk ditampilkan, sedangkan file CSS diakses berulang kali.
- **Program Counter (PC):** Alamat instruksi kode yang memicu permintaan data, sering digunakan dalam *hardware prefetching*.

2.4 Manajemen Cache Berbasis Flash

Penggunaan Solid State Drive (SSD) berbasis memori Flash sebagai tingkat *cache* (biasanya antara DRAM dan HDD/Storage Cloud) memperkenalkan tantangan baru yang tidak ditemukan pada manajemen memori berbasis DRAM. Bagian ini membahas isu kritis terkait keausan perangkat dan algoritma yang dirancang khusus untuk memitigasinya.

2.4.1 Karakteristik Fisik dan Write Endurance

Memori NAND Flash memiliki batasan fisik yang fundamental: setiap sel memori hanya dapat menahan jumlah siklus penghapusan dan penulisan (*Program/Erase cycles* atau P/E cycles) yang terbatas sebelum mengalami degradasi permanen.

- **Asimetri Operasi:** Membaca data dari Flash sangat cepat dan tidak merusak. Namun, menulis data membutuhkan tegangan tinggi yang secara perlahan merusak lapisan oksida pada transistor *floating-gate*. Terlebih lagi, data tidak dapat ditimpa secara langsung (*overwrite*); blok memori harus dihapus (*erase*) terlebih dahulu sebelum dapat ditulis ulang.
- **Endurance Terbatas:** Flash tipe SLC (*Single-Level Cell*) biasanya tahan hingga 100.000 siklus, sedangkan tipe MLC (*Multi-Level Cell*) dan TLC (*Triple-Level Cell*) yang lebih murah hanya tahan antara 3.000 hingga 10.000 siklus.

Sifat ini menjadikan operasi penulisan (Write) sebagai sumber daya yang sangat mahal pada SSD cache.

2.4.2 Fenomena Write Amplification (WA)

Dalam konteks SSD, jumlah data yang secara fisik ditulis ke chip Flash sering kali jauh lebih besar daripada jumlah data yang diminta untuk ditulis oleh sistem operasi. Fenomena ini disebut *Write Amplification* (WA).

$$WA = \frac{\text{Data Fisik Ditulis ke Flash}}{\text{Data Logis dari Host}}$$

WA disebabkan oleh mekanisme internal SSD seperti *Garbage Collection* (GC) dan *Wear Leveling*. Ketika *cache* penuh dan perlu menulis data baru, pengontrol SSD mungkin harus memindahkan data valid dari satu blok ke blok lain sebelum menghapus blok tersebut. Algoritma *cache* yang melakukan banyak penulisan acak kecil (*small random writes*) akan memicu WA yang sangat tinggi, mempercepat kematian perangkat.

2.4.3 Dampak Promosi Agresif pada SSD

Algoritma tradisional seperti LRU atau CLOCK dirancang untuk DRAM, di mana operasi "promosi" (memindahkan data ke posisi MRU) atau "sisipan" (memasukkan data baru ke *cache*) dianggap murah. Pada *Flash Cache*, setiap kali terjadi *cache miss* dan algoritma memutuskan untuk memasukkan data baru (promosi dari HDD ke SSD), satu operasi tulis (Write) terjadi.

- **Masalah LRU:** LRU secara naif memasukkan setiap objek yang diakses ke dalam *cache*. Jika beban kerja memiliki banyak objek yang hanya diakses sekali (*one-hit wonders*), LRU akan terus-menerus menulis data sampah ke SSD, menyebabkan tingkat keausan yang tidak perlu.
- **Churn Rate:** Tingkat pergantian isi *cache* yang tinggi tidak hanya menurunkan kinerja karena latensi penulisan, tetapi juga secara langsung membakar jatah umur SSD.

2.4.4 Strategi Mitigasi Keausan: Lazy Promotion

Untuk mengatasi masalah keausan, strategi *Lazy Promotion* diperkenalkan. Prinsip utamanya adalah: "Jangan menulis ke Flash kecuali objek tersebut terbukti berharga."

- **Filter pada Akses Pertama:** Pada akses pertama, objek hanya dicatat metadatanya di memori (DRAM) atau dimasukkan ke daftar hantu, tetapi datanya tidak ditulis ke SSD. Objek tersebut dilayani langsung dari HDD/Backend.
- **Promosi pada Akses Kedua:** Hanya jika objek tersebut diakses kembali dalam waktu singkat (*hit* pada daftar hantu), barulah objek tersebut dipromosikan (ditulis) ke SSD.

Strategi ini secara drastis mengurangi jumlah penulisan dengan memfilter *one-hit wonders*, yang merupakan mayoritas dari trafik pada beban kerja CDN dan Web.

2.4.5 Algoritma Flash-Aware

Beberapa algoritma telah dikembangkan spesifik untuk media Flash:

- **RIPQ (Relative Indexing Priority Queue):** Mengimplementasikan antrian prioritas pada Flash dengan menggunakan pendekatan log-structured. RIPQ menulis data baru dan data

yang dipromosikan secara sekuensial ke dalam blok aktif, yang meminimalkan Write Amplification. Ini menggabungkan akurasi tinggi dari antrian prioritas dengan pola akses tulis yang ramah Flash.

- **LARC (Lazy Adaptive Replacement Cache):** Varian dari ARC yang menambahkan mekanisme *lazy promotion*. LARC menggunakan ukuran filter dinamis untuk menyeimbangkan antara *hit ratio* dan *write count*, secara eksplisit mengorbankan sedikit *hit ratio* demi penghematan umur SSD yang signifikan.

2.5 Strategi Prefetching dan Spekulasi

Manajemen *cache* sering kali berjalan beriringan dengan mekanisme *prefetching*. Jika *cache* bertujuan mengelola data yang sudah diminta, *prefetching* bertujuan menebak data yang akan diminta di masa depan dan memuatnya sebelum dibutuhkan.

2.5.1 Klasifikasi Prefetcher

Berdasarkan lokasi implementasinya, *prefetcher* dapat dibagi menjadi:

1. **Hardware Prefetcher:** Beroperasi pada level kontroler memori atau L1/L2 cache. Biasanya menggunakan pola sederhana seperti *Stream Prefetcher* (mendeteksi akses berurutan) atau *Stride Prefetcher* (mendeteksi pola lompatan alamat yang konstan). Keunggulannya adalah kecepatan tinggi, namun terbatas pada pola akses memori fisik yang sederhana.
2. **Software Prefetcher:** Disisipkan oleh kompilator atau secara eksplisit oleh pemrogram (misalnya menggunakan instruksi `__builtin_prefetch` di C++). Ini memungkinkan penangkapan semantik aplikasi yang lebih kompleks, seperti traversal struktur data *linked list* atau *graph*.

2.5.2 Interaksi Cache dan Prefetcher

Hubungan antara *cache* dan *prefetcher* bersifat pedang bermata dua. Di satu sisi, *prefetching* yang akurat dapat menghilangkan *compulsory miss*. Namun, *prefetching* yang tidak akurat menyebabkan masalah serius:

- **Cache Pollution:** Data yang di-*prefetch* tetapi tidak pernah digunakan akan mengisi slot *cache*, berpotensi menggusur data berguna lainnya.
- **Bandwidth Waste:** Memuat data yang salah membuang siklus bus memori dan daya, yang sangat kritis pada sistem perangkat seluler.

Algoritma penggantian *cache* modern sering kali harus bersifat *prefetch-aware*, misalnya dengan menempatkan data hasil *prefetch* pada posisi LRU (segera dibuang jika tidak dipakai) alih-alih MRU.

2.6 Hierarki dan Arsitektur Penyimpanan Lanjut

Manajemen *cache* di dunia nyata jarang berdiri sendiri sebagai satu lapisan tunggal. Dalam sistem komputer modern, dari perangkat seluler hingga pusat data, penyimpanan diatur dalam hierarki yang kompleks.

2.6.1 Multi-Tier Storage (Penyimpanan Bertingkat)

Sistem penyimpanan perusahaan (*Enterprise Storage*) sering menggunakan tiga atau lebih tingkatan media penyimpanan:

1. **Tier 0 (Memory/DRAM):** Kecepatan tertinggi (nanodetik), kapasitas terkecil, dan harga termahal. Digunakan untuk *hot data* yang sangat kritis, seperti tabel indeks basis data atau metadata sistem berkas.
2. **Tier 1 (Performance/Flash SSD):** Menggunakan SSD NVMe atau SAS. Menawarkan keseimbangan antara performa (mikrodetik) dan kapasitas. Digunakan untuk data transaksi aktif.
3. **Tier 2 (Capacity/HDD/Tape):** Menggunakan Hard Disk Drive (HDD) kapasitas besar atau Tape. Latensi tinggi (milidetik) namun harga per GB sangat murah. Digunakan untuk arsip dan *backup*.

Tantangan utama dalam arsitektur ini adalah **Data Migration Policy**: kapan dan bagaimana memindahkan data antar-tier secara transparan tanpa mengganggu aplikasi. Algoritma seperti *Automated Tiering* memantau suhu data (*heat map*) dan memindahkan data secara periodik (misalnya tiap malam) atau real-time.

2.6.2 Exclusive vs Inclusive Caching

Dalam hierarki *cache* multi-level (misalnya L1, L2, L3 pada CPU, atau DRAM dan SSD pada storage), kebijakan inklusi menentukan duplikasi data:

- **Inclusive Cache:** Data di *cache* level atas (L1) dijamin juga ada di level bawah (L2). Ini menyederhanakan protokol koherensi karena jika data tidak ada di L2, pasti tidak ada di L1. Namun, ini memboroskan kapasitas efektif.
- **Exclusive Cache:** Data dijamin hanya ada di satu level. Jika L1 membuang data, data tersebut "diturunkan" (*demoted*) ke L2. Ini memaksimalkan kapasitas total hierarki, yang sangat penting untuk *storage cache* di mana rasio ukuran DRAM terhadap SSD sangat kecil Soundararajan2008Exclusivity.

2.7 Efisiensi Data dalam Storage Cache

Selain algoritma penggantian, efisiensi penyimpanan fisik data di dalam *cache* juga menjadi area penelitian aktif, terutama pada media mahal seperti Flash SSD.

2.7.1 Deduplication-Aware Caching

Banyak beban kerja penyimpanan mengandung data yang berulang (duplikat), seperti gambar OS virtual machine yang identik atau lampiran email yang sama. *Deduplikasi* menghilangkan salinan berlebih ini. Dalam konteks *cache*, **CacheDedup Li2016CacheDedup** mengusulkan arsitektur di mana *cache* menyimpan data unik berdasarkan konten (menggunakan *fingerprint* SHA-1), bukan berdasarkan alamat logis (LBA). Ini memungkinkan *cache* logis berukuran lebih besar daripada kapasitas fisiknya, meningkatkan *hit ratio* secara signifikan. Tantangannya adalah *overhead* komputasi untuk menghitung *hash* setiap blok data yang masuk.

2.7.2 Kompresi Data (Compression)

Kompresi (seperti LZ4 atau Zstd) dapat melipatgandakan kapasitas efektif *cache*. Namun, dekompresi berada di jalur kritis pembacaan data (*read path*), sehingga menambah latensi. Strategi *Selective Compression* mencoba mengompresi hanya data "dingin" di *cache*, sementara data "panas" disimpan dalam bentuk mentah untuk akses cepat.

2.8 Interaksi dengan Sistem Berkas dan IO

Perilaku *cache* juga sangat dipengaruhi oleh lapisan perangkat lunak di atasnya, yaitu sistem berkas (*file system*) dan manajer basis data.

2.8.1 Double Caching Problem

Sistem operasi modern (seperti Linux) menggunakan seluruh RAM yang tidak terpakai sebagai *Page Cache*. Jika aplikasi basis data (seperti PostgreSQL atau MySQL) juga memiliki *Buffer Pool* sendiri, terjadi fenomena *Double Caching*: data yang sama disimpan dua kali di RAM (sekali di user-space buffer, sekali di kernel page cache). Ini membuang memori. Solusinya adalah menggunakan **Direct I/O** (`O_DIRECT`) untuk membypass *cache* OS, menyerahkan kendali sepenuhnya pada aplikasi.

2.8.2 Journaling dan Log-Structured Writes

Sistem berkas *journaling* (seperti ext4 atau XFS) menulis metadata dua kali (ke jurnal dan ke lokasi akhir) untuk menjamin konsistensi saat listrik padam. Pola tulis ini dapat membingungkan algoritma *cache* pada level blok (SSD/HDD controller), karena metadata tampak sangat "panas". Algoritma *Semantic-Aware Caching* mencoba mengidentifikasi tipe data (apakah ini file log, metadata, atau data pengguna) untuk menerapkan kebijakan yang berbeda.

2.9 Isu Konkurensi dan Skalabilitas

Pada era komputasi *many-core*, skalabilitas struktur data manajemen *cache* menjadi hambatan utama (*bottleneck*).

2.9.1 Limitasi Skalabilitas LRU

Implementasi LRU standar menggunakan *doubly linked list* global. Setiap kali sebuah objek diakses (bahkan untuk operasi baca/Get), posisinya harus dipindahkan ke kepala antrian (MRU). Operasi pemindahan ini memerlukan penguncian (*locking*) global untuk menjaga konsistensi pointer. Akibatnya, pada sistem dengan puluhan *core* CPU yang mengakses *cache* secara bersamaan, terjadi *lock contention* yang parah. Kinerja sistem tidak lagi dibatasi oleh kecepatan memori, melainkan oleh waktu tunggu antrian kunci (*lock waiting time*).

2.9.2 Teknik Lock-Free dan Batching

Untuk mengatasi masalah ini, berbagai pendekatan dikembangkan:

- **CLOCK-based approximation:** Algoritma CLOCK atau Second Chance tidak memerlukan pemindahan node dalam memori, hanya mengubah bit penanda. Ini memungkinkan implementasi yang jauh lebih ramah konkurensi.
- **Batching Updates:** Algoritma seperti yang digunakan di Caffeine (Java cache library)

menggunakan *ring buffer* per-CPU untuk menampung event akses. Pembaruan struktur data global dilakukan secara *asynchronous* dan *batched*, sehingga jalur kritis pembacaan data (Get) bersifat *wait-free*.

- **Strictly Lock-Free Structures:** Menggunakan instruksi atomik (seperti CAS - Compare And Swap) untuk memanipulasi senarai, meskipun ini sering kali sangat kompleks untuk diimplementasikan dengan benar.

2.10 Arsitektur Caching Terdistribusi

Dalam skala *cloud* dan internet, *cache* jarang berdiri sendiri. Mereka beroperasi sebagai sistem terdistribusi yang masif (seperti Memcached atau Redis Cluster).

2.10.1 Konsistensi dan Hashing

Tantangan utama dalam *distributed caching* adalah bagaimana memetakan kunci objek ke server *cache* tertentu. Pendekatan *Modulo Hashing* sederhana ($hash(key) \bmod N$) memiliki kelemahan fatal: jika jumlah server N berubah (karena penambahan kapasitas atau kegagalan server), hampir seluruh kunci akan dipetakan ulang, menyebabkan badai *miss* (*thundering herd*).

Solusinya adalah **Consistent Hashing Karger1997Consistent**. Dalam skema ini, baik server maupun kunci dipetakan ke dalam sebuah cincin virtual (*hash ring*). Sebuah kunci dilayani oleh server terdekat dalam arah jarum jam di cincin tersebut. Jika sebuah server ditambahkan atau dihapus, hanya sebagian kecil kunci (tetangga dekat di cincin) yang perlu dipindahkan. Ini memungkinkan skalabilitas elastis yang dibutuhkan oleh layanan modern.

2.10.2 Protokol Koherensi Cache Terdistribusi

Ketika data yang di-*cache* diubah di sumber aslinya, salinan di berbagai node *cache* menjadi usang (*stale*). Terdapat beberapa strategi untuk menangani ini:

- **Time-to-Live (TTL):** Pendekatan konsistensi lemah (*eventual consistency*) di mana data dianggap valid selama periode waktu tertentu. Sederhana dan skalabel, namun berisiko menyajikan data lama.
- **Write-Through / Write-Back:** Pada *write-through*, data ditulis ke *cache* dan *backend* secara sinkron. Pada *write-back*, data ditulis ke *cache* dulu dan disinkronisasi ke *backend* belakangan. *Write-back* lebih cepat namun berisiko kehilangan data jika *cache* mati.
- **Cache Invalidation:** Sumber data mengirimkan sinyal ke seluruh *cache* untuk menghapus data tertentu saat terjadi perubahan. Ini menjamin konsistensi yang lebih kuat namun sulit diimplementasikan pada skala global karena masalah latensi jaringan.

2.11 Keamanan dan Side-Channel Attacks

Cache adalah sumber daya bersama (*shared resource*), yang membuka celah keamanan serius melalui serangan saluran samping (*side-channel attacks*).

2.11.1 Timing Attacks

Serangan seperti **Flush+Reload** atau **Prime+Probe** ^{Yarom2014}**FlushReload** mengeksploitasi perbedaan waktu akses antara *cache hit* dan *cache miss*. Penyerang dapat memantau set *cache* tertentu dan mengukur waktu aksesnya untuk menyimpulkan data apa yang sedang diakses oleh korban (misalnya kunci enkripsi RSA). Ini memaksa perancang sistem untuk mempertimbangkan isolasi *cache* (seperti *cache coloring* atau partisi) yang sering kali mengorbankan kinerja demi keamanan.

2.11.2 Cache Pollution Attacks

Penyerang dapat dengan sengaja membanjiri *cache* dengan permintaan objek acak (*random query flood*). Tujuannya adalah menggusur konten yang sah dan berguna, sehingga menurunkan kinerja sistem target (Denial of Service). Algoritma *cache* yang resisten terhadap serangan ini (seperti TinyLFU atau ARC) menjadi sangat penting dalam lingkungan publik.

2.12 Formalisasi Matematis Masalah Caching

Masalah manajemen *cache* dapat diformulasikan secara matematis sebagai masalah optimasi.

Misalkan kita memiliki kapasitas *cache* C dan urutan permintaan $R = \{r_1, r_2, \dots, r_N\}$. Tujuan kita adalah meminimalkan fungsi biaya total:

$$\min \sum_{t=1}^N \text{Cost}(r_t, S_t)$$

di mana S_t adalah keadaan *cache* pada waktu t . Biaya akses biasanya biner: 0 jika $r_t \in S_t$ (hit) dan 1 jika tidak (miss). Aturan transisi keadaan mengharuskan $|S_t| \leq C$.

Untuk algoritma *Offline Optimal* (Belady), masalah ini dapat dimodelkan sebagai masalah *Min-Cost Flow* pada graf. Untuk kasus ukuran objek dan biaya pengambilan yang bervariasi (General Caching), masalah ini menjadi NP-Hard, dan sering kali didekati dengan relaksasi menjadi masalah *Knapsack* fraksional atau algoritma aproksimasi seperti *Landlord*.

2.13 Graph Neural Networks (GNN) untuk Caching

Pendekatan terbaru dalam ML-Caching melibatkan representasi relasional antar objek menggunakan graf. Banyak data di dunia nyata memiliki struktur graf implisit: halaman web saling bertautan via *hyperlink*, video di YouTube memiliki keterkaitan rekomendasi.

GNN dapat digunakan untuk mempelajari *embedding* dari setiap objek yang menangkap topologi relasi tersebut. Alih-alih hanya melihat sejarah akses temporal (seperti LSTM), GNN memungkinkan model memprediksi akses masa depan berdasarkan kedekatan struktural di dalam graf. Misalnya, jika pengguna mengakses halaman A, maka halaman B yang terhubung erat dengan A dalam graf pengetahuan memiliki probabilitas tinggi untuk diakses, dan GNN dapat merekomendasikan *prefetching* B bahkan sebelum pola akses tersebut muncul dalam sejarah linear.

2.14 Green Caching: Efisiensi Energi

Dalam era IoT dan pusat data hemat energi, *caching* tidak hanya soal kinerja, tetapi juga konsumsi daya. Memori SRAM (Static RAM) yang digunakan untuk L1/L2 cache sangat cepat tetapi boros daya karena kebocoran arus (*leakage current*). Teknologi memori baru seperti STT-RAM (Spin-Transfer Torque RAM) menawarkan kepadatan tinggi dan kebocoran nol (karena non-volatile), namun memiliki latensi tulis dan energi tulis yang lebih tinggi.

Algoritma *Green Caching* bertujuan menyeimbangkan trade-off ini dengan teknik seperti:

- **Cache Way-Shutdown:** Mematikan sebagian bank *cache* saat beban kerja rendah.
- **Hybrid Cache Architectures:** Menempatkan data yang sering ditulis di SRAM dan data yang hanya dibaca di STT-RAM untuk meminimalkan konsumsi energi total.

[Halaman ini sengaja dikosongkan]

BAB III

METODOLOGI

Bab ini merinci metodologi yang digunakan dalam penelitian ini, mulai dari datasets, perancangan dan pengembangan perangkat lunak simulasi, proses pengumpulan dan pengolahan data, hingga implementasi dan evaluasi model *machine learning*.

3.1 Datasets

Dalam penelitian ini, penulis menggunakan dataset berupa *trace* sintesis yang mengikuti distribusi Zipfian. Pemilihan distribusi Zipfian ini didasarkan pada karakteristiknya yang sangat menyerupai pola akses pada *web trace* di dunia nyata, di mana sebagian kecil konten diakses jauh lebih sering dibandingkan sisanya (popularitas yang tidak merata) **Breslau1999Web**.

Seluruh dataset dibuat secara prosedural menggunakan utilitas skrip dari pustaka `libCacheSim`. Pendekatan ini dipilih penulis karena menawarkan fleksibilitas yang lebih baik dibandingkan menggunakan *log server* mentah. Dengan menggunakan data sintesis, penulis dapat menghilangkan *noise* atau gangguan yang sering terdapat pada data riil, serta memiliki kontrol penuh terhadap parameter distribusi untuk mensimulasikan berbagai kondisi beban kerja yang spesifik.

Secara struktural, dataset ini disimpan menggunakan format `oracleGeneral`. Dalam format ini, data disusun sebagai urutan akses di mana setiap entri merepresentasikan satu permintaan. Komponen utama dalam struktur ini adalah kolom `object_id`, yang berisi identifikasi numerik unik untuk setiap objek yang diminta. Format ini memastikan kompatibilitas langsung dengan simulator tanpa memerlukan langkah pra-pemrosesan yang kompleks.

Untuk keperluan eksperimen, penulis membuat beberapa set *trace* Zipfian yang berbeda secara terpisah. Pembagian antara data untuk pelatihan (*training*) dan pengujian (*testing*) dilakukan secara ketat dan berbeda file. Hal ini krusial untuk memastikan bahwa evaluasi performa model dilakukan pada pola akses baru yang belum pernah dilihat sebelumnya. Tujuannya adalah untuk menguji kemampuan generalisasi sistem dalam menangani trafik, bukan sekadar kemampuan model dalam menghafal urutan data latih.

3.1.1 Parameterisasi Distribusi

Untuk merepresentasikan variasi lokalitas temporal yang realistis, penulis menggunakan parameter kemiringan (*skewness parameter*) α yang bervariasi. Dalam distribusi Zipfian, probabilitas akses ke objek ke- i didefinisikan sebagai $P(i) \propto 1/i^\alpha$. Penelitian ini menggunakan 1 sebagai α . Nilai α yang lebih tinggi mengindikasikan konsentrasi popularitas yang lebih ekstrem pada sejumlah kecil objek (*heavy tail*), yang merupakan karakteristik umum pada beban kerja *Content Delivery Network* (CDN) modern.

3.1.2 Statistik Beban Kerja

Setiap *trace* yang dibangkitkan memiliki karakteristik volume yang seragam untuk menjaga konsistensi pengujian. Rincian statistik dari dataset sintesis yang digunakan disajikan dalam Tabel 3.1. Ukuran *footprint* yang besar dipilih untuk menekan mekanisme penggan-

tiap *cache* agar bekerja di bawah tekanan (*cache pressure*) yang tinggi, sehingga perbedaan performa antar algoritma dapat terlihat lebih signifikan.

Tabel 3.1: Spesifikasi Dataset Sintetis Zipfian

Parameter	Nilai / Keterangan
Total Permintaan (<i>Requests</i>)	10.000.000
Objek Unik (<i>Unique Objects</i>)	1.000.000
Parameter Zipf (α)	1.0
Format Trace	oracleGeneral (Biner)
Ukuran Data Per Entri	16 bytes (Time + ID + Size)
Sifat Distribusi	Stasioner (<i>Stationary</i>)

3.1.3 Stasioneritas Data

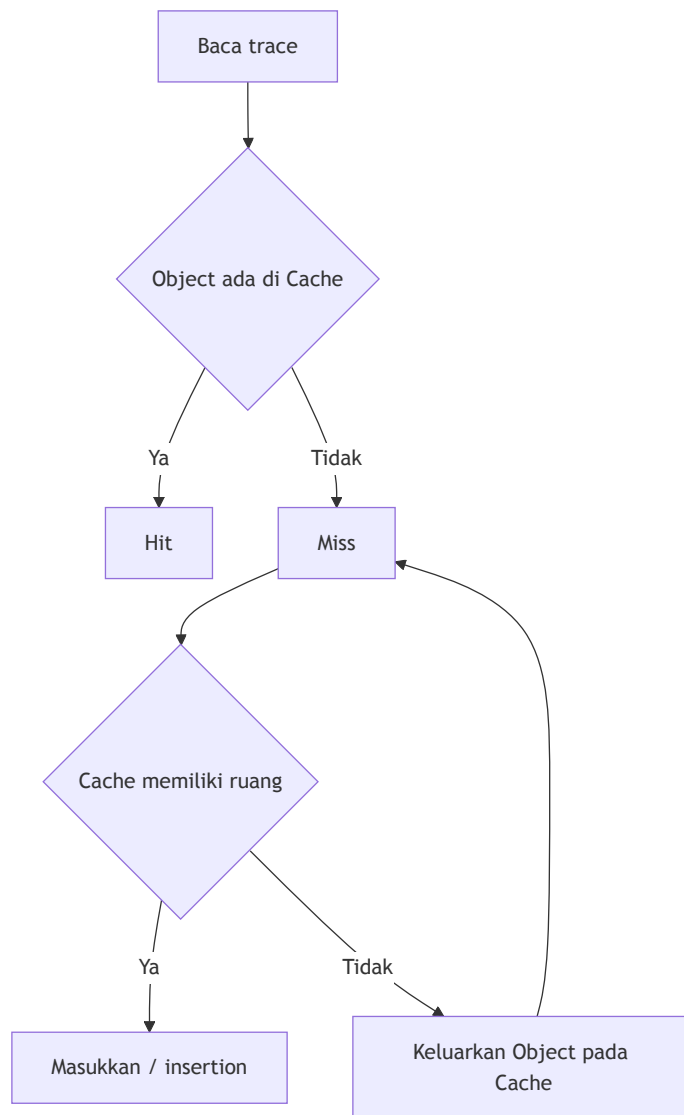
Penting untuk dicatat bahwa dataset sintetis yang digunakan dalam penelitian ini bersifat stasioner. Artinya, probabilitas popularitas setiap objek tidak berubah sepanjang waktu simulasi. Hal ini dilakukan secara sengaja untuk mengisolasi variabel performa algoritma dalam kondisi ideal sebelum menghadapi dinamika perubahan fase (*phase change*) yang mungkin terjadi pada data riil.

3.2 Pengembangan Perangkat Lunak Simulasi

Langkah pertama dalam penelitian ini adalah pengembangan perangkat lunak simulasi kustom yang mampu memenuhi kebutuhan analisis. Perangkat lunak ini dirancang dengan beberapa persyaratan utama:

1. **Integrasi Model Machine Learning:** Perangkat lunak harus mampu memanggil dan menjalankan model *machine learning* secara efisien di dalam fungsi *eviction* dari algoritma *cache*.
2. **Dukungan Algoritma Pembanding:** Selain algoritma yang diusulkan, perangkat lunak harus dapat menjalankan algoritma CLOCK standar dan Offline CLOCK sebagai tolak ukur perbandingan kinerja.
3. **Pengumpulan Data dari Offline CLOCK:** Perangkat lunak harus mampu menjalankan mode Offline CLOCK untuk mengumpulkan data-data krusial, antara lain:
 - (a) Nilai *miss ratio* dan jumlah promosi untuk setiap iterasi simulasi.
 - (b) Metadata dari setiap objek pada saat keputusan promosi dibuat, beserta label apakah promosi tersebut pada akhirnya sia-sia (*wasted*) atau tidak.
 - (c) Kemampuan untuk mengeksport semua data yang terkumpul ke dalam format CSV (*Comma-Separated Values*) untuk memfasilitasi proses pengolahan dan analisis data lebih lanjut.

Untuk mempercepat proses pengembangan, penulis memanfaatkan **LibCacheSim**, sebuah pustaka sumber terbuka yang dirancang khusus untuk pengembangan simulator *cache*. Pustaka ini ditulis dalam bahasa C/C++, yang menjamin kinerja komputasi yang sangat efisien, terutama untuk tugas-tugas berat seperti pengumpulan data dari *trace* berukuran besar.



Gambar 3.1: Diagram alur dari program simulasi yang dikembangkan.

3.3 Pengumpulan Data dan Labelling

Pengumpulan data awal dilakukan sepenuhnya menggunakan perangkat lunak simulasi yang telah dikembangkan. Penulis menjalankan simulasi ini dengan tujuan utama untuk mendapatkan metrik kinerja dasar seperti jumlah promosi total dan *miss ratio* yang dihasilkan oleh algoritma CLOCK standar. Angka-angka ini berfungsi sebagai *baseline* untuk mengukur efektivitas model yang diusulkan nantinya.

Bersamaan dengan simulasi baseline tersebut, penulis melakukan proses pembuatan dataset yang akan digunakan untuk melatih model *machine learning*. Dataset ini disusun dengan cara mencatat (*logging*) metadata relevan dari setiap objek tepat pada saat objek tersebut menjadi kandidat untuk keputusan promosi. Fitur-fitur yang dicatat meliputi statistik akses masa lalu dan status objek dalam antrian saat itu.

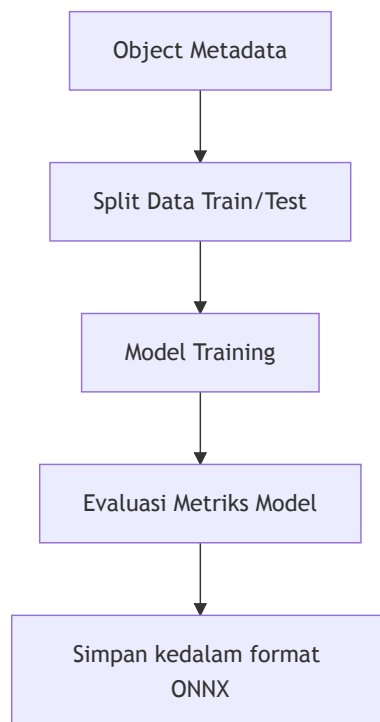
Pemberian label pada setiap sampel data dilakukan dengan merujuk pada simulasi *Of-*

fine CLOCK yang dijalankan setelahnya. Karena *Offline CLOCK* memiliki visibilitas terhadap seluruh *trace* masa depan, algoritma ini dapat menentukan keputusan optimal secara retrospektif. Penulis memberikan label *wasted* pada objek yang dipromosikan tetapi tidak memberikan *hit* sebelum akhirnya dikeluarkan (*evict*) kembali dari *cache*. Sebaliknya, label *not wasted* diberikan jika promosi tersebut terbukti berguna dan menghasilkan *hit*.

3.4 Pengembangan Model Machine Learning

Berdasarkan dataset terlabel yang telah disiapkan sebelumnya, penulis mengembangkan sebuah model *machine learning* sebagai mesin inferensi utama. Model ini didesain secara spesifik untuk memproses vektor fitur yang diekstraksi dari metadata objek secara *real-time* tanpa membebani latensi sistem. Variabel masukan yang digunakan meliputi frekuensi akses historis dan durasi waktu sejak akses terakhir atau *recency*.

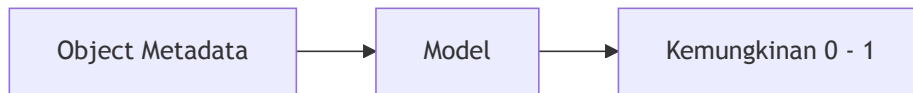
Sebagai keluaran, model menghasilkan skor probabilitas kontinu dengan rentang nilai antara 0 hingga 1. Dalam konteks klasifikasi biner ini, nilai probabilitas ditafsirkan sebagai tingkat keyakinan terhadap label *wasted*. Artinya, nilai keluaran yang mendekati 1 mengindikasikan prediksi kuat bahwa objek tersebut tidak akan memberikan manfaat atau *hit* jika dipromosikan ke *cache* utama. Sebaliknya, nilai yang rendah menandakan bahwa objek tersebut memiliki potensi utilitas yang tinggi dan layak untuk mendapatkan promosi.



Gambar 3.2: Diagram proses pelatihan model *machine learning*.

3.5 Implementasi Model pada Perangkat Lunak Simulasi

Setelah tahap pelatihan selesai, model *machine learning* yang dihasilkan diintegrasikan kembali ke dalam perangkat lunak simulasi untuk pengujian kinerja. Tantangan teknis utama



Gambar 3.3: Arsitektur model *machine learning* yang digunakan.

pada tahap ini adalah menjembatani perbedaan lingkungan antara proses pelatihan yang berbasis Python dengan lingkungan simulasi yang ditulis dalam C++ demi efisiensi memori dan waktu eksekusi. Solusi yang diterapkan penulis adalah dengan mengekspor model tersebut ke dalam format **ONNX (Open Neural Network Exchange)**. Format ini dipilih karena standar interoperabilitasnya yang tinggi, memungkinkan model yang dilatih pada *framework* tingkat tinggi untuk dijalankan pada lingkungan C++ tanpa perlu menulis ulang logika model secara manual.

Eksekusi model di dalam simulator dilakukan menggunakan **ONNX Runtime**, sebuah *inference engine* yang dioptimalkan untuk kinerja tinggi dan latensi rendah. Secara arsitektural, modul inferensi ini ditanamkan langsung ke dalam logika internal algoritma CLOCK, spesifiknya pada fungsi yang menangani mekanisme *eviction* dan promosi. Setiap kali sistem mempertimbangkan sebuah objek untuk dipromosikan ke segmen memori utama, simulator secara *real-time* mengekstrak metadata objek tersebut dan mengubahnya menjadi tensor input yang sesuai dengan spesifikasi model ONNX.

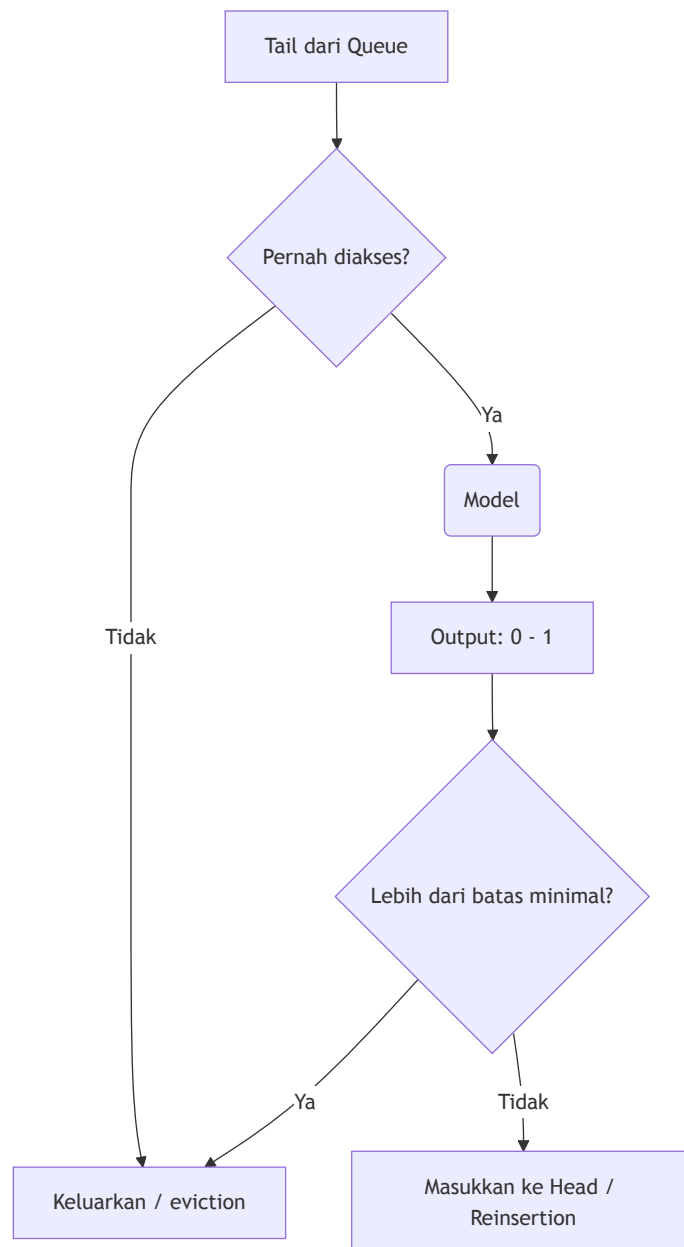
Keputusan akhir ditentukan berdasarkan perbandingan antara keluaran model dengan nilai ambang batas (*threshold*) sensitivitas yang telah dikonfigurasi sebelumnya. Model menghasilkan skor probabilitas yang merepresentasikan kecenderungan objek untuk menjadi *wasted*. Jika skor probabilitas ini melampaui ambang batas, sistem akan mengambil tindakan preventif dengan menggagalkan promosi dan langsung mengusir objek tersebut (*bypass*). Mekanisme ini bertujuan untuk mencegah polusi *cache* oleh objek-objek yang diprediksi tidak akan memberikan keuntungan kinerja di masa depan.

3.6 Evaluasi Model

Tahap final dari rangkaian penelitian ini adalah evaluasi komprehensif terhadap kinerja model yang telah diintegrasikan ke dalam simulator. Untuk mengukur efektivitas solusi yang diusulkan, penulis melakukan perbandingan langsung (*benchmarking*) antara algoritma CLOCK yang dimodifikasi dengan tiga standar referensi utama.

Referensi pertama adalah algoritma CLOCK standar tanpa modifikasi. Ini berfungsi sebagai *baseline* untuk melihat peningkatan kinerja riil dibandingkan metode konvensional yang ada saat ini. Referensi kedua adalah *Offline CLOCK*, sebuah algoritma teoretis yang diasumsikan memiliki pengetahuan sempurna akan masa depan. Perbandingan ini krusial untuk mengetahui seberapa dekat kinerja model prediksi dengan batas optimal yang mungkin dicapai (*upper bound*).

Referensi ketiga adalah strategi *Optimal Zipf Promotion*. Metode ini memanfaatkan pengetahuan *oracle* mengenai distribusi popularitas global dataset. Dalam strategi ini, promosi hanya diberikan kepada sebagian kecil objek yang berada di puncak kurva popularitas Zipfian (objek-objek terpopuler). Perbandingan ini bertujuan untuk memverifikasi apakah strategi berbasis popularitas statis sudah cukup efisien, atau apakah pendekatan dinamis berbasis *machine learn-*



Gambar 3.4: Diagram alur implementasi model pada simulator.

ing mampu menangkap pola akses yang lebih kompleks yang terlewatkan oleh metrik popularitas semata.

Metrik evaluasi difokuskan pada dua indikator utama. Indikator pertama adalah *miss ratio*, yang merepresentasikan efisiensi dasar *cache* dalam melayani permintaan pengguna. Indikator kedua adalah jumlah total promosi ke segmen utama. Metrik ini menjadi fokus khusus karena berkorelasi lurus dengan operasi tulis (*write operations*). Keberhasilan model didefinisikan sebagai kemampuannya menekan jumlah promosi secara signifikan untuk menghemat *bandwidth* dan umur perangkat, tanpa menyebabkan degradasi yang berarti pada *miss ratio*.

3.7 Perangkat Keras dan Lunak

Penelitian ini didukung oleh infrastruktur komputasi dan tumpukan perangkat lunak sebagai berikut:

3.7.1 Perangkat Keras

Testbed Utama PC server berkinerja tinggi yang ditenagai oleh prosesor Intel Xeon Silver 4114 (40 *logical cores* @ 3.00 GHz) dan memori RAM sebesar 192 GiB. Perangkat ini didedikasikan untuk menjalankan simulasi paralel yang intensif dan memuat dataset *trace* berukuran besar ke dalam memori.

Unit Pengembangan Lokal Laptop pribadi yang digunakan untuk tahap pengembangan kode awal, *prototyping*, serta analisis hasil pasca-simulasi.

3.7.2 Perangkat Lunak

C++23 Bahasa pemrograman utama untuk implementasi simulator. Dipilih untuk memaksimalkan efisiensi eksekusi waktu nyata dan kontrol memori tingkat rendah.

Python 3.13 Bahasa skrip yang digunakan untuk orkestrasi eksperimen, pemrosesan data awal (*preprocessing*), serta pelatihan model *machine learning*.

libCacheSim Pustaka simulator *cache* yang menjadi fondasi dasar pengembangan sistem, menyediakan implementasi struktur data *cache* yang efisien.

ONNX Runtime Mesin inferensi (*inference engine*) berkinerja tinggi yang digunakan untuk menjalankan model prediksi di dalam lingkungan C++ dengan latensi minimal.

Pandas & Plotly Pustaka Python yang digunakan untuk analisis statistik dataset dan visualisasi data hasil eksperimen.

OpenSSH Digunakan untuk akses jarak jauh, manajemen *job*, dan transfer data antara unit lokal dan *testbed*.

[Halaman ini sengaja dikosongkan]

BAB IV

PENGUJIAN DAN ANALISIS

Bagian ini menguraikan hasil evaluasi kuantitatif dari model ML-CLOCK yang diusulkan. Kinerja model dibandingkan dengan tiga standar: CLOCK Standar (Baseline), Optimal Zipf Promotion (Heuristik Oracle Sederhana), dan Offline CLOCK (Batas Atas Teoretis). Pengujian dilakukan pada empat variasi ukuran *cache*: 1%, 10%, 20%, dan 40% dari total ukuran dataset unik.

4.1 Perbandingan Kinerja Keseluruhan

Tabel 4.1 menyajikan rangkuman lengkap kinerja seluruh metode. Nilai negatif pada *Delta Miss Ratio* menandakan perbaikan (penurunan miss ratio), sedangkan nilai positif menandakan degradasi kinerja (kenaikan miss ratio).

Tabel 4.1: Perbandingan Komprehensif: ML-CLOCK vs. Referensi

Ukuran Cache	Metode	Delta Miss Ratio	Reduksi Promosi
1%	ML-CLOCK (Usulan)	-0.05%	10%
	Optimal Zipf	-0.20%	10%
	Offline CLOCK	-0.35%	40%
10%	ML-CLOCK (Usulan)	-0.10%	10%
	Optimal Zipf	-0.90%	13%
	Offline CLOCK	-1.80%	43%
20%	ML-CLOCK (Usulan)	-0.20%	10%
	Optimal Zipf	-1.67%	15%
	Offline CLOCK	-3.50%	43%
40%	ML-CLOCK (Usulan)	+0.15% (Regresi)	8%
	Optimal Zipf	-3.30%	17%
	Offline CLOCK	-8.00%	43%

4.2 Analisis Karakteristik Algoritma Pembanding

Untuk menempatkan kinerja ML-CLOCK dalam perspektif yang objektif, karakteristik fundamental dari ketiga algoritma pembanding harus dipahami sebagai berikut:

4.2.1 Optimal Zipf Promotion (Oracle Heuristik)

Strategi ini bertindak sebagai algoritma *Oracle* yang "curang" karena memanfaatkan **pengetahuan masa depan yang lengkap** mengenai distribusi data. Algoritma ini secara eksplisit

membaca identitas objek (`objectId`) dan hanya mempromosikan objek yang diketahui berada di puncak kurva popularitas Zipfian. Dalam skenario dunia nyata, sistem tidak memiliki akses ke "kunci jawaban" berupa distribusi probabilitas global ini. Oleh karena itu, Optimal Zipf memberikan batas kinerja heuristik yang sangat tinggi namun tidak praktis untuk implementasi nyata.

4.2.2 Offline CLOCK (Batas Atas Teoretis)

Offline CLOCK merupakan representasi dari kesempurnaan teoretis (mirip dengan algoritma Belady's OPT). Algoritma ini bekerja dengan cara memindai seluruh *trace* akses di masa depan sebelum simulasi dimulai. Keputusan promosi diambil dengan logika deterministik sempurna: sebuah objek hanya akan dipromosikan jika dan hanya jika objek tersebut dipastikan akan diakses kembali (*hit*) sebelum waktu pengusurannya tiba. Kesenjangan kinerja yang masif antara ML-CLOCK dan Offline CLOCK (43% reduksi promosi) menunjukkan batas fisik maksimal yang bisa dicapai jika prediksi masa depan memiliki akurasi 100%.

4.2.3 CLOCK Standar (Baseline)

Algoritma ini mewakili pendekatan konvensional tanpa kecerdasan buatan. Ia bekerja secara reaktif murni berdasarkan prinsip *Recency* (kebaruan). Kelemahan fatalnya adalah ketidakmampuan membedakan objek populer dengan objek yang hanya lewat sekali (*one-hit wonders*). Akibatnya, CLOCK Standar sering melakukan promosi yang sia-sia (*wasted promotions*), yang memboroskan siklus penulisan memori dan mencemari *cache* dengan data sampah.

4.3 Evaluasi Model Machine Learning

Meskipun kalah dari strategi Oracle dan Offline yang memiliki pengetahuan masa depan, model ML-CLOCK menunjukkan karakteristik kinerja yang spesifik sebagai solusi *online*:

1. **Efektivitas Terbatas pada Cache Kecil:** Pada ukuran 1% hingga 20%, model berhasil memberikan kontribusi positif meskipun marjinal. Penurunan *miss ratio* sebesar 0.05% – 0.20% membuktikan bahwa model mampu belajar membedakan objek *wasted*.
2. **Kegagalan Generalisasi pada Cache Besar:** Pada ukuran 40%, terjadi regresi kinerja yang signifikan. Ambang batas (*threshold*) klasifikasi model mungkin kurang adaptif untuk kapasitas penyimpanan yang sangat besar.
3. **Peran Vital Bobot dan Threshold:** Tanpa penerapan bobot *loss* kecil (0.5) untuk kelas *wasted* dan ambang batas prediksi tinggi (0.6 – 0.8), model menjadi terlalu agresif menolak promosi (*aggressive eviction*). Mekanisme kontrol ini mencegah model membuang terlalu banyak objek potensial yang akan menghancurkan kinerja cache (penurunan *miss ratio*).

4.4 Analisis Fitur dan Konfigurasi Model

Model terbaik yang dihasilkan menggunakan kombinasi fitur frekuensi (`lifetime_freq`, `cache_freq`) dan kebaruan (`vtime_since_access`). Penting dicatat bahwa konfigurasi ambang batas tinggi (0.6 – 0.8) berfungsi sebagai mekanisme pertahanan (*safeguard*). Model memiliki bias bawaan untuk mengklasifikasikan objek sebagai *wasted*. Tanpa ambang batas

yang ketat ini, sistem akan mengalami *over-eviction*, di mana penghematan promosi mungkin melonjak drastis, namun dibayar mahal dengan penurunan akurasi (peningkatan *miss ratio*).

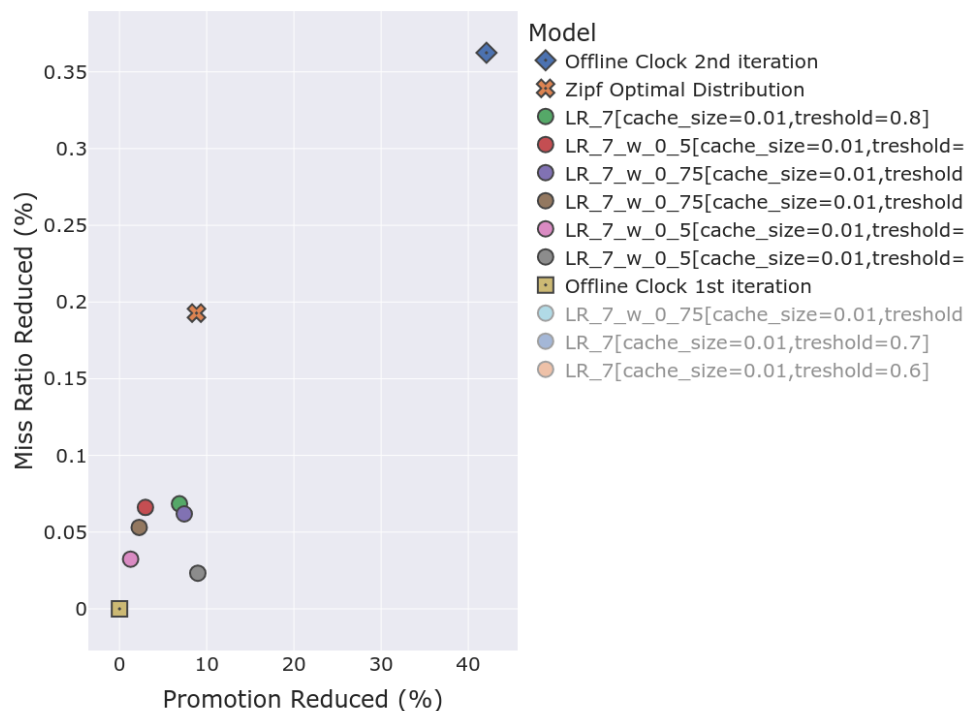
Bagian ini akan membedah dua perilaku ekstrem dari ML-CLOCK berdasarkan konfigurasi bobot pelatihan (*training weights*). Data eksperimen membuktikan bahwa tanpa intervensi manual pada *loss function*, model ML cenderung menjadi prediktor yang "malas" dan bias terhadap kelas mayoritas, yang berujung pada hasil yang buruk.

4.4.1 Model Konservatif (Weighted Loss)

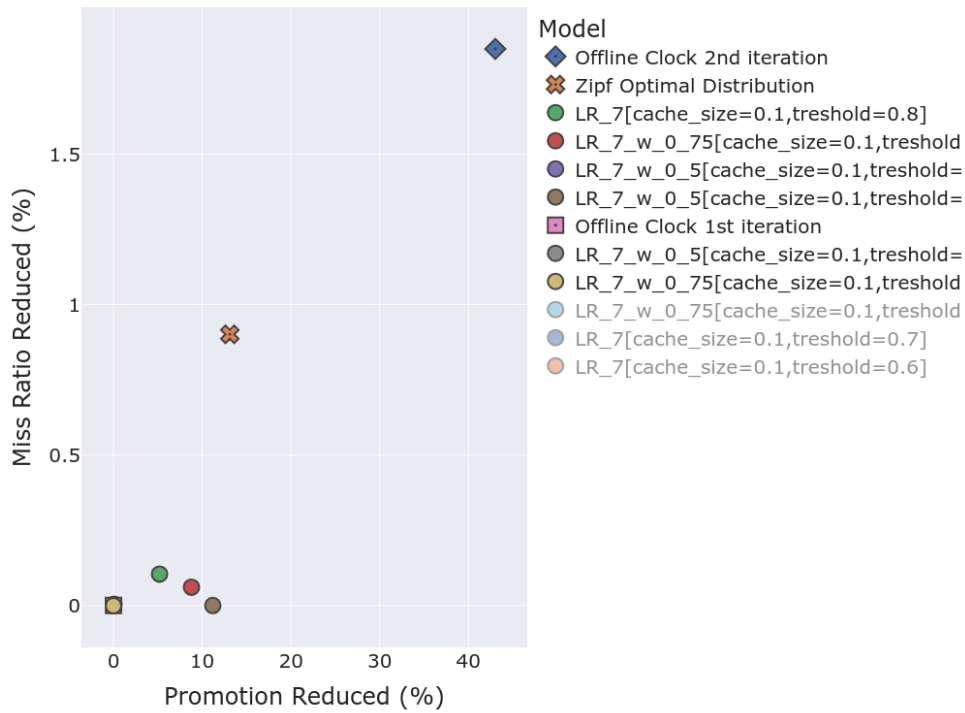
Varian ini dilatih menggunakan *custom loss weight* yang memberikan penalti berat (rasio 2:1 atau lebih tinggi) jika model melakukan kesalahan fatal berupa membuang objek populer (*False Positive*) yang akan meningkatkan miss ratio secara signifikan.

Karakteristik kinerja yang teramati pada Gambar ?? hingga Gambar ?? adalah sebagai berikut:

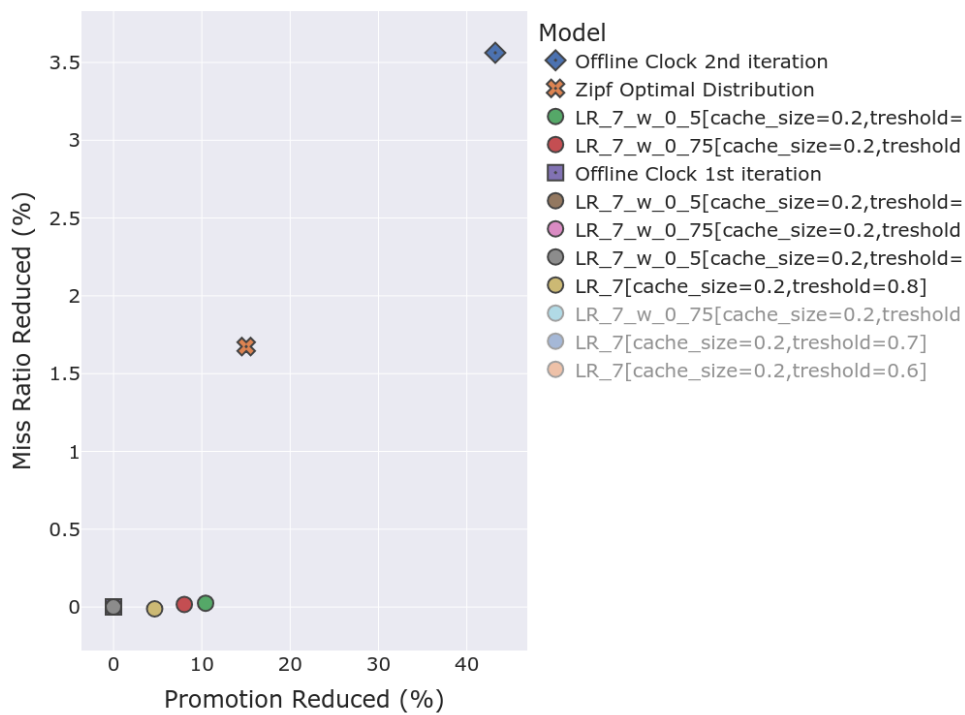
1. **Mendekati CLOCK:** Hasil dari model mendekati CLOCK standar. Ini adalah indikator positif bahwa model berhasil menginternalisasi heuristik dasar *Recency*.
2. **Efisiensi Terbatas:** Konsekuensinya, reduksi promosi terbatas di angka 10%. Ini adalah harga yang harus dibayar. Model dipaksa untuk menahan diri dalam membuang objek, sehingga banyak objek yang sebenarnya tidak berguna (*borderline*) tetap lolos dan ditulis ke Cache.



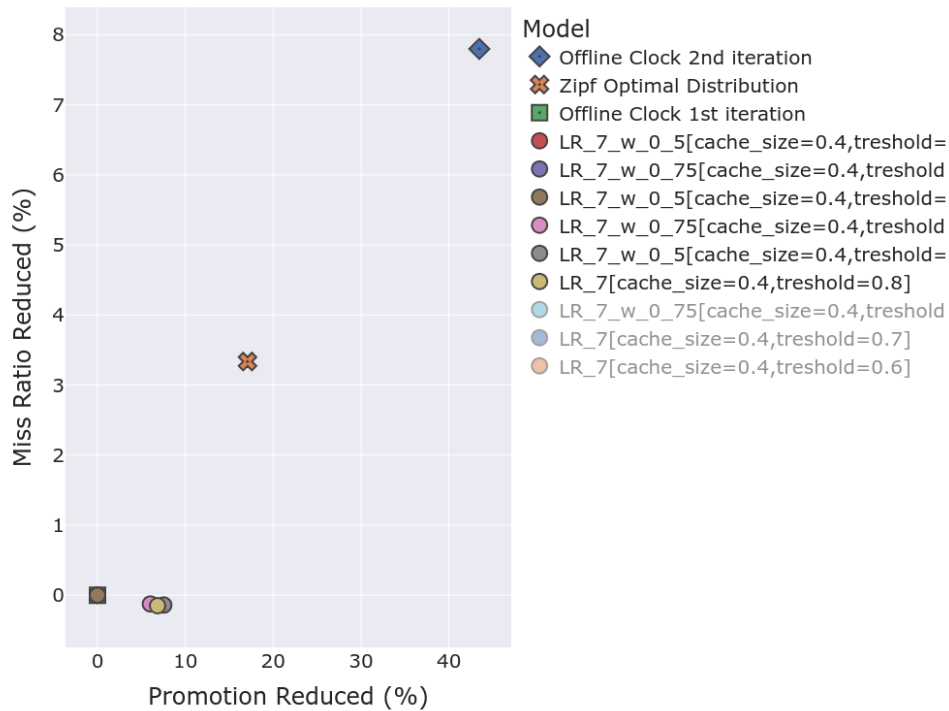
Gambar 4.1: Model Konservatif pada ukuran cache 1%



Gambar 4.2: Model Konservatif pada ukuran cache 10%



Gambar 4.3: Model Konservatif pada ukuran cache 20%



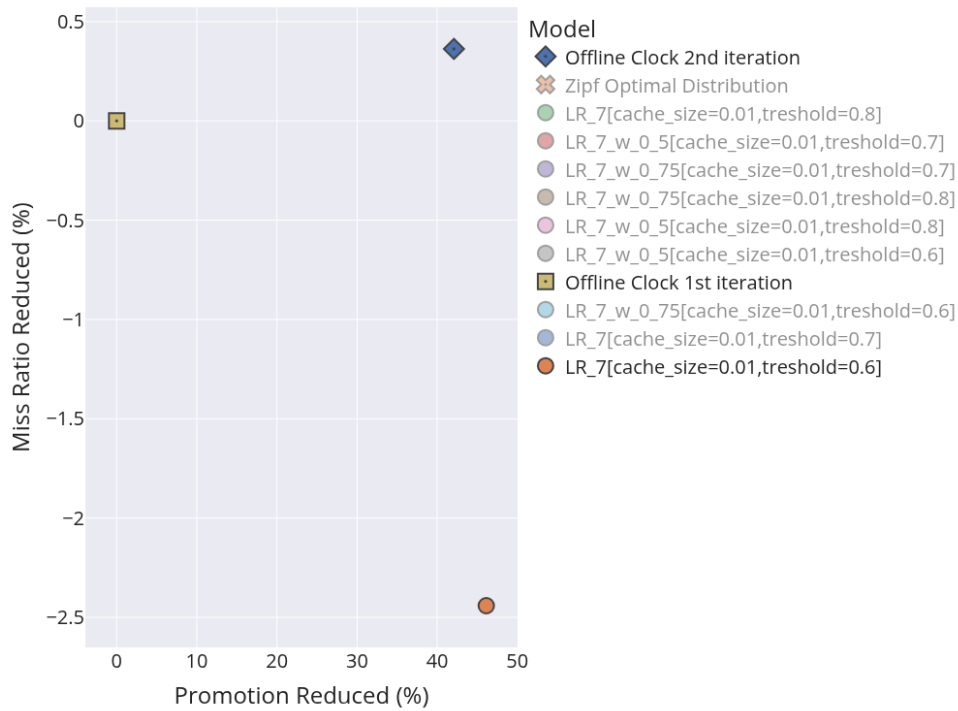
Gambar 4.4: Model Konservatif pada ukuran cache 40%

4.4.2 Model Agresif (Unweighted Loss)

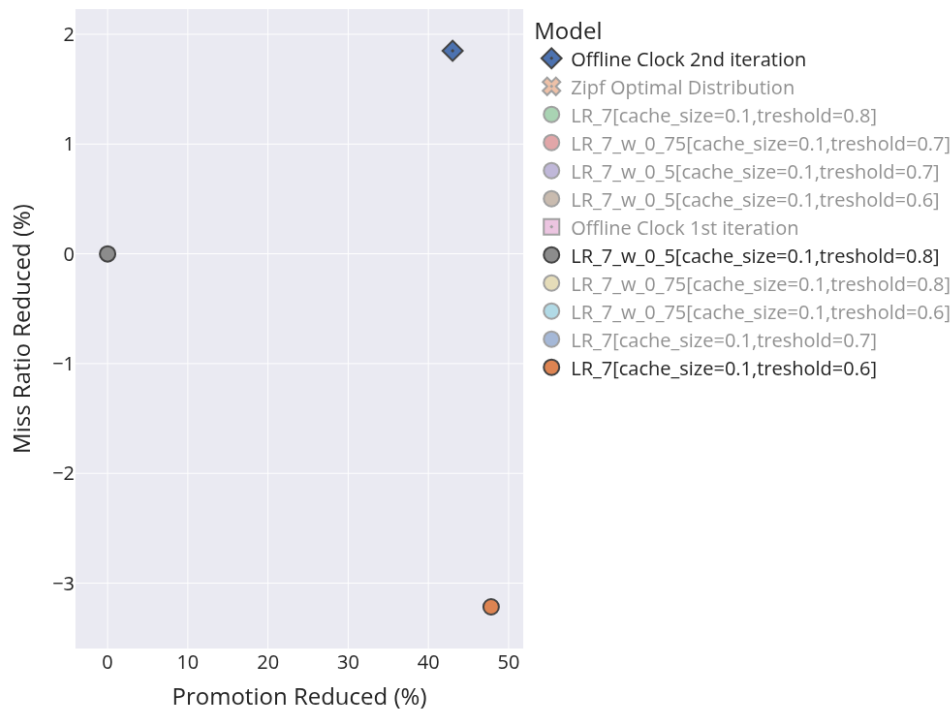
Model ini dilatih secara naif tanpa pembobotan kelas (*unweighted*). Membiarkan fungsi optimasi ditentukan oleh statistik data latih yang timpang. Karena mayoritas objek dalam *trace* Zipfian adalah "sampah" (akses sekali), model belajar bahwa strategi paling aman untuk meminimalkan *global error* adalah dengan memprediksi "sampah" sesering mungkin.

Analisis pada Gambar ?? hingga ?? menunjukkan kegagalan sistemik dari strategi ini:

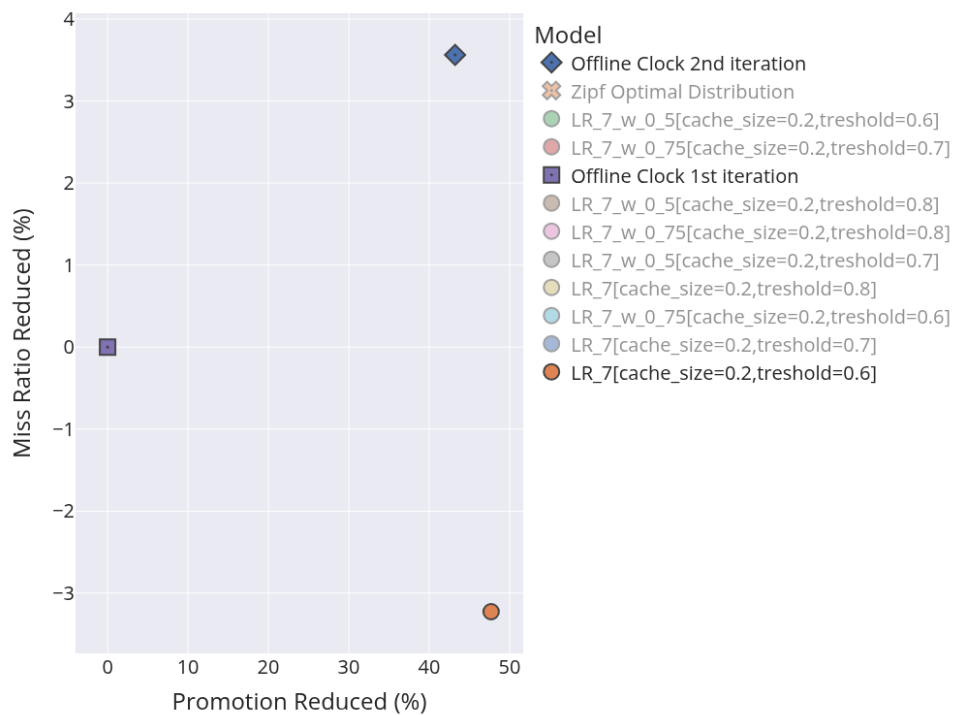
1. **Degradasi Layanan (Miss Penalty):** Terjadi lonjakan *miss ratio* (gap negatif) rata-rata 2% dibandingkan CLOCK. Dalam sistem skala *petabyte*, degradasi 2% bukan angka kecil. Hal ini menerjemahkan jutaan I/O tambahan ke *backend storage* yang lambat, meningkatkan latensi rata-rata sistem secara signifikan.



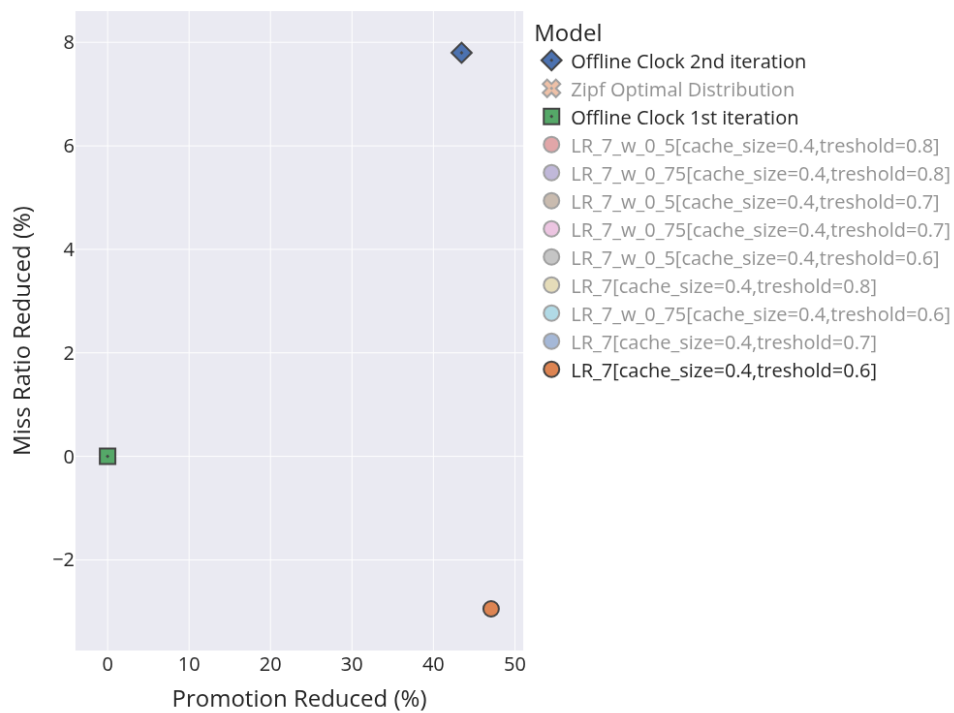
Gambar 4.5: Model Agresif pada ukuran cache 1%



Gambar 4.6: Model Agresif pada ukuran cache 10%



Gambar 4.7: Model Agresif pada ukuran cache 20%



Gambar 4.8: Model Agresif pada ukuran cache 40%

4.4.3 Analisis Akar dari Perbedaan Kedua Model

Perbedaan radikal antara kedua model ini berakar pada masalah klasik *imbalanced dataset*. Dalam distribusi Zipfian ($\alpha \approx 1.0$), rasio objek yang hanya diakses sekali (*wasted*) terhadap objek yang diakses berulang (*reused*) bisa mencapai 80:20.

Tanpa mekanisme *rebalancing* seperti *weighted loss*, model ML akan terjebak dalam minimum lokal trivial. Model menemukan bahwa dengan selalu memprediksi kelas mayoritas (*wasted*), ia bisa mencapai akurasi dasar 80% tanpa perlu mempelajari pola fitur yang kompleks. Ini menghasilkan model yang memiliki *Recall* sangat rendah untuk kelas negatif (*not_wasted* / objek populer). Akibatnya, saat objek populer masuk, model gagal mengidentifikasinya dan langsung membuangnya, menyebabkan *cache thrashing* yang parah.

4.4.4 Penerapan dari Kedua Model

Pemilihan antara model konservatif dan agresif pada akhirnya adalah keputusan ekonomi, bukan sekadar teknis.

1. **Biaya Latensi Lebih Besar:** Untuk aplikasi sensitif latensi (seperti e-commerce atau perbankan), biaya yang timbul akibat *cache miss* (kehilangan pelanggan, transaksi lambat) jauh melampaui biaya penggantian SSD. Oleh karena itu, Model Konservatif adalah satu-satunya pilihan rasional.
2. **Biaya Hardware Lebih Besar:** Model Agresif hanya masuk akal dalam skenario *edge case* di mana media penyimpanan sangat murah namun memiliki ketahanan tulis yang sangat rendah (seperti QLC NAND kelas konsumen) dan aplikasi tidak peduli pada latensi. Namun, untuk kasus seperti itu, algoritma deterministik sederhana seringkali lebih efisien daripada model ML yang berat.

BAB V

PENUTUP

Bab ini merangkum temuan-temuan kunci yang diperoleh dari eksperimen integrasi *machine learning* ke dalam algoritma manajemen *cache* CLOCK, serta memberikan rekomendasi strategis untuk pengembangan penelitian selanjutnya.

5.1 Kesimpulan

Berdasarkan analisis data pengujian dan evaluasi kinerja yang telah dipaparkan pada bab sebelumnya, penulis menarik beberapa kesimpulan kritis sebagai berikut:

1. **Efisiensi Penulisan Terbukti Namun Terbatas.** Implementasi ML-CLOCK berhasil membuktikan hipotesis awal bahwa model prediktif dapat mengurangi operasi penulisan (*promotions*) ke memori utama. Pengurangan konsisten sebesar 8–10% tercapai di seluruh variasi ukuran *cache*. Meskipun angka ini signifikan untuk memperpanjang umur perangkat penyimpanan (*endurance*), capaian ini masih jauh di bawah batas optimal teoretis *Offline CLOCK* yang mencapai 43%. Hal ini menunjukkan bahwa model masih bersikap konservatif dalam pengambilan keputusan.
2. **Trade-off Kinerja pada Skala Besar.** Terdapat pola kinerja non-linear yang jelas. Pada lingkungan dengan tekanan memori tinggi (ukuran *cache* 1% – 20%), ML-CLOCK memberikan keuntungan ganda: pengurangan *miss ratio* dan penghematan promosi. Namun, terjadi regresi kinerja pada ukuran *cache* 40%, di mana model menjadi terlalu selektif (*over-selective*) sehingga membuang objek yang seharusnya disimpan saat ruang tersedia. Ini menegaskan bahwa pendekatan ML statis tanpa adaptasi dinamis terhadap ketersediaan ruang memiliki risiko pada kapasitas besar.
3. **Superioritas Oracle Mengungkap Batas Prediksi Lokal.** Dominasi strategi *Optimal Zipf Promotion* terhadap ML-CLOCK di seluruh metrik mengkonfirmasi bahwa dalam distribusi Zipfian, pengetahuan global mengenai popularitas objek jauh lebih berharga daripada pola akses lokal (*recency* dan frekuensi jangka pendek). ML-CLOCK yang hanya mengandalkan metadata lokal kesulitan untuk mereplikasi akurasi yang dimiliki oleh algoritma berbasis pengetahuan global tersebut.
4. **Kontribusi Arsitektur Integrasi ONNX.** Terlepas dari kinerja model prediksi spesifik yang digunakan, kontribusi teknis utama dari penelitian ini adalah keberhasilan pembangunan kerangka kerja (*framework*) simulasi hibrida. Mekanisme integrasi model Python ke dalam lingkungan C++ berkinerja tinggi menggunakan format **ONNX (Open Neural Network Exchange)** terbukti andal, stabil, dan minim latensi. Infrastruktur ini memecahkan hambatan bahasa pemrograman yang selama ini menyulitkan peneliti sistem operasi untuk mengadopsi model *deep learning* modern.

5.2 Saran

Penelitian ini membuka jalan bagi eksplorasi lebih lanjut dalam domain *AI-driven cache replacement*. Untuk mengatasi keterbatasan yang ditemukan, penulis mengajukan saran-saran berikut:

1. **Pemanfaatan Kerangka Kerja ONNX untuk Riset Lanjutan.** Arsitektur simulasi yang telah dibangun dengan integrasi *ONNX Runtime* dirancang agar bersifat modular dan agnostik terhadap jenis model. Peneliti lain sangat disarankan untuk menggunakan kembali (*reuse*) infrastruktur ini untuk menguji arsitektur model yang berbeda (seperti LSTM, Transformer, atau Reinforcement Learning) tanpa perlu membangun ulang simulator dari nol. Kode sumber simulator dapat dijadikan basis standar (*testbed*) untuk komunitas riset manajemen memori.
2. **Penggunaan Trace Dunia Nyata (Real-world Workloads).** Evaluasi pada dataset sintetis Zipfian memiliki keterbatasan dalam menangkap dinamika temporal yang kompleks, seperti *burstiness* atau perubahan fase popularitas. Penelitian selanjutnya disarankan menggunakan *trace* produksi dari CDN (Content Delivery Network) atau basis data skala besar. Data riil akan memberikan tantangan yang lebih adil bagi model ML dibandingkan distribusi statis Zipfian yang cenderung menguntungkan metode heuristik sederhana.
3. **Eksplorasi Fitur Non-Linear.** Fitur *vtime_since_access* terbukti kurang efektif dan berpotensi menjadi *noise* pada distribusi Zipfian. Disarankan untuk mengeksplorasi fitur yang lebih kompleks, seperti distribusi waktu antar-kedatangan (*inter-arrival time distribution*) atau fitur berbasis grafik relasi antar-objek, untuk menangkap pola akses yang tidak terdeteksi oleh statistik frekuensi sederhana.
4. **Mekanisme Threshold Adaptif.** Untuk mengatasi masalah regresi pada *cache* ukuran besar, model statis dengan ambang batas tetap (0.6 – 0.8) harus ditinggalkan. Disarankan pengembangan mekanisme *adaptive thresholding* di mana ambang batas penerimaan objek diturunkan secara otomatis ketika tekanan pada memori rendah (banyak ruang kosong) dan dinaikkan ketika memori penuh, menyerupai perilaku kontrol umpan balik (*feedback loop*).

DAFTAR PUSTAKA

[Halaman ini sengaja dikosongkan]

BIOGRAFI PENULIS



Penulis, **Muhammad Haekal Muhyidin Al-Araby**, lahir di [Tempat Lahir] pada tanggal [Tanggal Lahir]. Penulis merupakan anak dari pasangan [Nama Ayah] dan [Nama Ibu].

Penulis menempuh pendidikan dasar di [Nama SD] dan lulus pada tahun [Tahun Lulus SD]. Kemudian, penulis melanjutkan pendidikan menengah pertama di [Nama SMP] dan lulus pada tahun [Tahun Lulus SMP]. Setelah itu, penulis menyelesaikan pendidikan menengah atas di [Nama SMA/SMK] pada tahun [Tahun Lulus SMA/SMK].

Pada tahun [Tahun Masuk Kuliah], penulis diterima sebagai mahasiswa di Program Studi Teknik Komputer, Departemen Teknik Komputer, Fakultas Teknologi Elektro dan Informatika Cerdas, Institut Teknologi Sepuluh Nopember (ITS) melalui jalur [Jalur Masuk, contoh: SNMPTN/SBMPTN/Mandiri].

Selama masa perkuliahan, penulis memiliki minat khusus pada bidang [Sebutkan Minat Bidang Anda, contoh: rekayasa perangkat lunak, kecerdasan buatan, sistem komputer]. Untuk memperdalam minat tersebut, penulis aktif dalam [Sebutkan kegiatan, contoh: penelitian di laboratorium, organisasi, atau proyek pribadi].

Untuk memenuhi salah satu syarat kelulusan dan memperoleh gelar Sarjana Teknik, penulis menyusun Tugas Akhir yang berjudul "PENERAPAN MODEL MACHINE LEARNING UNTUK MENINGKATKAN EFISIENSI ALGORITMA CLOCK PADA CACHE REPLACEMENT" di bawah bimbingan Bapak Reza Fuad Rachmadi, S.T., M.T., Ph.D dan Bapak Dr. Arief Kurniawan, S.T., M.T.

Penulis dapat dihubungi melalui surel di [Alamat Email Anda]. Email Anda].

[Halaman ini sengaja dikosongkan]